



LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Sviluppo di un sistema RAG aziendale e valutazione di LLM locali

Francesco Romeo
Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**

Co-relatore: **Dott. Marco Loregian**

Anno Accademico **2025/2026**

Sommario

“abstract”

Indice

Introduzione	iii
1 Fondamenti Tecnologici del Progetto RAG Aziendale	1
1.1 Architetture Transformer	1
1.1.1 Dalle reti neurali ricorrenti ai Transformer	1
1.1.2 Il meccanismo di Self-Attention	2
1.1.3 Architettura encoder-decoder e varianti decoder-only	2
1.1.4 Limiti in contesto zero-shot e motivazione del RAG	2
1.2 LLM: panorama e analisi comparativa	3
1.2.1 Modelli proprietari	3
1.2.2 Motivazioni per l'adozione di modelli locali	4
1.2.3 Modelli open-weight per uso on-premise	5
1.2.4 Inferenza locale nel prototipo aziendale	5
1.2.5 Ollama: server di inferenza locale	5
1.3 Retrieval-Augmented Generation (RAG)	6
1.3.1 Motivazione e definizione	6
1.3.2 Architettura del sistema RAG	6
1.3.3 Modelli di embedding	7
1.3.4 Database vettoriali	8
1.3.5 Agentic RAG	8
1.3.6 Tecniche di adattamento al dominio	9
1.4 Framework di orchestrazione: Haystack	10
1.5 Sintesi e posizionamento del lavoro	10
2 Analisi dei Requisiti e Progettazione del Sistema	11
2.1 Il contesto aziendale: Intré S.r.l.	11
2.1.1 Il modello delle Gilde	12
2.2 Caso d'uso: Proposta di nuove Gilde Intré	12
2.2.1 Motivazione del caso d'uso	13
2.2.2 Obiettivi del caso d'uso	13
2.2.3 Descrizione dei dati e analisi del corpus	13
2.3 Requisiti di sistema	14
2.3.1 Vincoli architetturali	15
2.4 Scelte tecnologiche: analisi comparativa	15
2.4.1 Framework di orchestrazione RAG	16

2.4.2	Database vettoriale	17
2.4.3	Modello di embedding	18
2.4.4	Server di inferenza LLM e selezione del modello di chat	18
2.5	Proposta architetturale	19
2.5.1	Componenti principali	19
2.5.2	Progettazione della collection Qdrant	21
2.5.3	Pipeline di ingestione ETL	22
2.5.4	Pipeline di retrieval e logica di ranking	22
2.5.5	Endpoint API di riferimento	23
2.6	Integrazione con i3Guild	23
2.6.1	Servizi Docker aggiuntivi	24
2.6.2	Variabili d'ambiente	24
2.6.3	Integrazione lato Next.js	24
2.6.4	Sincronizzazione dei dati	25
3	Implementazione del Sistema RAG	26
3.1	Struttura dei repository e responsabilità	27
3.2	Pipeline di ingestione	27
3.2.1	Configurazione e controlli preliminari	28
3.2.2	Estrazione dei dati da PostgreSQL	28
3.2.3	Normalizzazione e costruzione dei documenti	29
3.2.4	Embedding e scrittura in Qdrant	29
3.3	Microservizio RAG	29
3.3.1	Avvio, middleware e gestione errori	29
3.3.2	Endpoint di retrieval	30
3.3.3	Pipeline Haystack di query	30
3.4	Logica di retrieval	31
3.4.1	Riscrittura della query e filtri	31
3.4.2	Cache e deduplicazione	31
3.4.3	Selezione dinamica dei documenti	31
3.5	Generazione con Ollama	32
3.5.1	Risoluzione del modello	32
3.5.2	Costruzione del prompt aumentato	32
3.5.3	Streaming NDJSON	32
3.6	Workflow agentico	32
3.7	Integrazione lato Next.js	33
3.7.1	Client di retrieval	33
3.7.2	Client chat e generazione controllata	33
3.8	Generazione della bozza di Gilda	33
3.9	Aspetti operativi	34
3.9.1	Timeout e dipendenze lente	34
3.9.2	System prompt e modello derivato	34
3.9.3	Tracciabilità	35
3.10	Sintesi del capitolo	35
	Conclusioni e Sviluppi Futuri	36

A	Annotazioni e Convenzioni	37
A.1	Repository e componenti	37
A.2	Formato dei documenti indicizzati	37
A.3	Eventi NDJSON	37
B	Codice Utilizzato	39
B.1	Listati richiamati nel Capitolo 2	39
B.2	Listati richiamati nel Capitolo 3: pipeline di ingestione	40
B.3	Listati richiamati nel Capitolo 3: retrieval e generazione	42
B.4	Listati richiamati nel Capitolo 3: frontend e generazione bozza	43
	Bibliografia	45
	Ringraziamenti	48

Introduzione

- **Contesto:** Diffusione dei Large Language Models (LLM) e le sfide dell'adozione in ambito corporate (privacy, allucinazioni, aggiornamento dei dati). → giustificare le affermazioni con citazioni e dati.
- **Il Problema:** Gestione della conoscenza non strutturata e valorizzazione del materiale formativo (Gilde) in Intré. → descrizione dell'azienda e dello use-case.
- **Obiettivo della tesi:** Progettazione e sviluppo di un sistema RAG (Retrieval-Augmented Generation) con focus sull'ottimizzazione dell'inferenza e dell'accuratezza nel dominio specifico.
- **Estrema sintesi dei risultati ottenuti**
- **NOTA:** essere esplicativi sotto tutti i punti di vista, deve essere una versione sintetica della tesi - un riassunto - con l'aggiunta delle motivazioni del progetto.

Todo list

Capitolo 1

Fondamenti Tecnologici del Progetto RAG Aziendale

Il sistema sviluppato durante il progetto aziendale nasce da un'esigenza applicativa precisa: rendere interrogabile la conoscenza prodotta in i3Guild senza trasferire dati interni verso servizi esterni. Prima di descrivere il caso d'uso, i requisiti e l'implementazione, è quindi necessario introdurre i concetti tecnici minimi su cui si fonda la soluzione: modelli linguistici generativi, inferenza locale, *Retrieval-Augmented Generation* (RAG), embedding, database vettoriali e orchestrazione della pipeline.

Il capitolo non ha l'obiettivo di esaurire lo stato dell'arte sull'inferenza locale o sulla quantizzazione. Questi temi diventano oggetto specifico dei Capitoli ?? e ?. Qui vengono presentati solo nella misura necessaria a comprendere il prototipo realizzato in azienda e le sue scelte progettuali.

1.1 Architetture Transformer

1.1.1 Dalle reti neurali ricorrenti ai Transformer

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [29], costituisce il punto di svolta da cui derivano gli attuali modelli linguistici. Prima della sua introduzione, gran parte dei sistemi di trattamento del linguaggio naturale (*Natural Language Processing*, NLP) si basava su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, in particolare, su architetture LSTM (*Long Short-Term Memory*) [14]. Tali modelli elaboravano la sequenza un elemento alla volta, aggiornando progressivamente uno stato nascosto. Questa impostazione era coerente con la natura ordinata del testo, ma introduceva due limiti rilevanti: la scarsa parallelizzabilità dell'addestramento e la difficoltà nel preservare dipendenze informative tra elementi molto distanti nella sequenza (*vanishing gradient*).

Il Transformer affronta tali limiti sostituendo la ricorrenza con un meccanismo di *self-attention* globale. In questo modo ogni token può essere messo in relazione diretta con tutti gli altri token della sequenza, indipendentemente dalla loro distanza posizionale. Questa scelta architetturale spiega sia la maggiore efficienza in addestramento sia la capacità dei modelli successivi di costruire rappresentazioni contestuali più ricche.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model} , si calcolano tre proiezioni lineari per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione [29] è definito come:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (1.1)$$

dove d_k è la dimensione dei vettori di chiave; il fattore $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari molto grandi.

Il medesimo calcolo viene poi replicato su più *teste* di attenzione (*attention heads*). Ciascuna testa apprende una proiezione diversa dello spazio di input e può quindi specializzarsi su relazioni differenti, ad esempio sintattiche, semantiche o posizionali [29]. La concatenazione degli output di h teste definisce il meccanismo di *Multi-Head Attention*:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (1.2)$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza non è catturata dall'attention; viene iniettata tramite *positional encoding*, ossia vettori sommati agli embedding di input che forniscono informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder. L'encoder trasforma la sequenza di input in una rappresentazione contestuale densa; il decoder utilizza tale rappresentazione, tramite *cross-attention*, per produrre la sequenza di output. Questa struttura è particolarmente adatta a compiti *sequence-to-sequence*, come la traduzione automatica [29].

I modelli linguistici generativi contemporanei adottano prevalentemente un'architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token dipende esclusivamente dai precedenti. Questo vincolo è necessario per l'addestramento autoregressivo, nel quale il modello impara a predire il token successivo dato il contesto.

1.1.4 Limiti in contesto zero-shot e motivazione del RAG

La capacità dei Transformer di modellare il linguaggio non elimina tuttavia un problema centrale per l'uso aziendale: un LLM pre-addestrato non conosce, per definizione, i dati interni prodotti dopo l'addestramento e non dispone di un meccanismo nativo per verificarli. In un dominio come quello considerato in questa tesi, dove le risposte devono riferirsi a documenti aziendali specifici, questa caratteristica diventa un limite operativo. La Tabella 1.1 riassume le criticità principali e le relative strategie di mitigazione.

Da questa analisi deriva la scelta di privilegiare il paradigma RAG, discusso nella Sezione 1.3. Rispetto al fine-tuning, RAG permette di aggiornare la conoscenza disponibile intervenendo sul corpus documentale e non sui pesi del modello; per questo risulta più adatto a basi di conoscenza aziendali soggette a modifiche frequenti.

Tabella 1.1: Limiti strutturali degli LLM in contesti aziendali e relative strategie di mitigazione.

Limitazione	Descrizione	Strategia di mitigazione
Conoscenza statica	I parametri codificano la conoscenza fino al <i>knowledge cutoff</i> ; nessun accesso a dati interni o aggiornati dopo l'addestramento.	RAG: recupero dinamico da knowledge base esterna, aggiornabile senza riaddestrare il modello.
Allucinazioni [18]	Il modello può generare testo plausibile ma fattualmente errato su informazioni assenti dalla distribuzione di addestramento.	RAG con <i>grounding</i> : la risposta è ancorata a documenti recuperati e verificabili.
Privacy dei dati	Il fine-tuning su modelli cloud può esporre dati riservati a infrastrutture di terze parti, con rischi di conformità normativa.	Esecuzione on-premise con modelli open-weight: nessuna trasmissione di dati verso servizi esterni.

1.2 LLM: panorama e analisi comparativa

Una volta chiariti i limiti dei modelli generativi, occorre distinguere tra le famiglie di LLM disponibili. A partire dal 2022 l'ecosistema si è diviso lungo una linea rilevante per questa tesi: da un lato i modelli proprietari, accessibili prevalentemente tramite API cloud; dall'altro i modelli open-weight, i cui pesi possono essere scaricati ed eseguiti su infrastruttura locale, nel rispetto delle relative licenze. La Tabella 1.2 offre una panoramica dei modelli considerati come riferimento tecnologico.

Tabella 1.2: Panorama comparativo dei principali LLM (2024–2025). [†] Esecuzione garantita on-premise senza dipendenze cloud; [‡] pesi scaricabili pubblicamente; MoE = *Mixture of Experts*.

Modello	Sviluppatore	Licenza	Accesso	Architettura / Param. attivi	Ctx	Multi-modale
GPT-5 / o3	OpenAI	Proprietaria	API cloud	Dense, n.d.	400k	✓
Claude 4 Sonnet	Anthropic	Proprietaria	API cloud	Dense, n.d.	200k	✓
Gemini 3 Pro	Google DM	Proprietaria	API cloud	Dense, n.d.	1M	✓
Llama 4 Scout	Meta AI	Open-weight [‡]	Self-hosted [†]	MoE, 17B	10M	✓
Mistral Large 3	Mistral AI	Apache 2.0 [‡]	Self-hosted [†]	MoE, 41B	256k	✓
Qwen3-8B	Alibaba Cloud	Apache 2.0 [‡]	Self-hosted [†]	Dense, 8B	128k	–

1.2.1 Modelli proprietari

I modelli proprietari offrono prestazioni elevate sui benchmark generali, ma presentano vincoli rilevanti per l'adozione in contesti con requisiti di riservatezza.

GPT-5 e OpenAI o-series. La famiglia GPT-5 [49] rappresenta un riferimento recente per pianificazione complessa, sviluppo software e integrazione multimodale. La serie "o" (o1, o3) è

invece orientata a compiti che richiedono ragionamento deliberato, con miglioramenti osservati in ambiti scientifici e logico-matematici [50]. L'erogazione è vincolata ad API cloud proprietarie; in ambito enterprise è possibile operare tramite servizi dedicati, ad esempio Azure OpenAI Service, che offrono garanzie contrattuali di isolamento dei dati.

Claude 4 (Anthropic). I modelli Claude adottano approcci come la *Constitutional AI* [3], un protocollo di addestramento guidato da principi per migliorare l'allineamento. Anche questi modelli sono erogati in modalità cloud; Anthropic propone modalità contrattuali (ad esempio *Zero Data Retention*) per rispondere a esigenze enterprise [35].

Gemini (Google DeepMind). La famiglia Gemini [43] è caratterizzata da multimodalità nativa e da finestre di contesto molto estese, fino a un milione di token per Gemini 3 Pro. Anche in questo caso l'accesso è fornito tramite servizi cloud.

1.2.2 Motivazioni per l'adozione di modelli locali

Nel caso in esame, le prestazioni assolute non sono l'unico criterio di scelta. Il sistema descritto in questa tesi deve poter operare su dati aziendali interni e deve quindi privilegiare controllo, tracciabilità e assenza di dipendenze da servizi esterni. Per questo l'architettura utilizza modelli eseguiti interamente in locale (*self-hosted*). La scelta si fonda su tre motivazioni, sintetizzate nella Tabella 1.3.

Tabella 1.3: Motivazioni per l'adozione di modelli LLM on-premise.

Motivazione	Descrizione	Riferimento normativo / letteratura
Sovranità del dato e conformità	L'esecuzione locale evita la trasmissione di dati verso infrastrutture di terze parti, anche in presenza di garanzie contrattuali (ZDR, DPA).	GDPR [11]; rischi del cloud nella gestione di dati sensibili.
Costi operativi nel lungo periodo	Il pricing a consumo dei provider cloud può risultare più oneroso rispetto all'ammortamento dell'hardware locale in scenari ad alto volume.	[5]; rischio di <i>vendor lock-in</i> .
Controllo e personalizzazione	I modelli open-weight (LLaMA, Mistral, Qwen) consentono fine-tuning supervisionato e allineamento su dati propri, senza le restrizioni delle policy dei sistemi proprietari.	[28, 19, 24]

Per queste ragioni, i modelli proprietari sono mantenuti come riferimento comparativo, mentre l'analisi progettuale si concentra sulle soluzioni eseguibili localmente. Tale impostazione rende coerente la scelta del modello con i vincoli di privacy, costo e governo dell'infrastruttura discussi nei capitoli successivi.

1.2.3 Modelli open-weight per uso on-premise

Llama 4 (Meta AI). Rilasciata nell'aprile 2025, la famiglia Llama 4 [46] adotta architetture *Mixture of Experts* (MoE) e supporta la multimodalità. Le varianti *Scout* e *Maverick* bilanciano parametri effettivi e numero di esperti per gestire contesti estesi fino a 10 milioni di token. La licenza open-weight presenta clausole specifiche da valutare in fase di adozione, in particolare per operatori e per l'Unione Europea.

Mistral 3 (Mistral AI). Nel 2025 Mistral AI ha ampliato la propria offerta con modelli orientati all'integrazione locale [47]. Il modello di punta, *Mistral Large 3*, è un modello MoE multilingue; la famiglia include varianti ottimizzate per l'esecuzione su hardware locale. La distribuzione sotto licenza Apache 2.0 facilita l'impiego in progetti aperti.

Qwen3 (Alibaba Cloud). La generazione Qwen3 [57] introduce modalità operative differenziate per task logici ed efficienza. La famiglia copre modelli da dimensioni molto ridotte fino a varianti MoE con finestre di contesto estese. Nel Capitolo 3 si descrive la scelta della variante 8B per il sistema proposto, motivata dal bilancio tra qualità semantica e velocità di inferenza su CPU-only.

1.2.4 Inferenza locale nel prototipo aziendale

L'esecuzione locale di un LLM introduce un vincolo ulteriore rispetto all'uso di API cloud: il modello deve convivere con database, servizi applicativi, pipeline RAG e runtime di inferenza nello stesso ambiente controllato. Nel progetto i3Guild questo requisito deriva dalla necessità di non esporre dati aziendali a servizi esterni. La conseguenza pratica è che la scelta del modello non può basarsi solo sulla qualità linguistica, ma deve considerare memoria disponibile, latenza di caricamento, tempo al primo token e throughput.

La quantizzazione viene quindi introdotta qui come motivazione tecnica generale: ridurre la precisione dei pesi può diminuire l'occupazione di memoria e rendere eseguibili modelli altrimenti troppo costosi. Tuttavia, l'effetto reale dipende dal runtime, dal formato del modello e dal costo di dequantizzazione. Per questo l'analisi sistematica della quantizzazione viene separata dal progetto aziendale e ripresa nei capitoli finali, dedicati all'inferenza locale su Apple Silicon.

1.2.5 Ollama: server di inferenza locale

Alla luce dei vincoli descritti, il progetto adotta un'infrastruttura di inferenza locale gestita tramite **Ollama** [48], un server open source che espone modelli locali in formato GGUF tramite un'API REST compatibile con la specifica *OpenAI Chat Completions*. Ollama semplifica il ciclo di vita dei modelli locali: l'installazione avviene tramite un comando semplice (`ollama pull <modello>`), la gestione del contesto e del formato di prompt è automatizzata, e il server può essere distribuito come container Docker. Nel contesto di questo progetto, Ollama serve il modello di chat; la generazione degli embedding è invece affidata a una pipeline separata basata su FastEmbed e Haystack, come descritto nella Sezione 1.3.3.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [21], risponde al limite degli LLM come sistemi a conoscenza chiusa (*closed-book*). In un modello generativo tradizionale, infatti, la conoscenza disponibile è codificata nei parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, la generazione viene preceduta da una fase di *retrieval* dinamico. Prima di invocare l’LLM, il sistema interroga una base di conoscenza esterna e recupera i documenti più pertinenti rispetto alla query. Tali documenti vengono poi inseriti nel prompt (*prompt augmentation*), così che la risposta sia fondata non solo sulla conoscenza parametrica del modello, ma anche su fonti aggiornate e verificabili [26]. Questa separazione tra modello e conoscenza esterna è il motivo per cui RAG si adatta bene a domini aziendali in evoluzione.

1.3.2 Architettura del sistema RAG

Dal punto di vista architetturale, un sistema RAG separa il momento in cui la conoscenza viene preparata dal momento in cui viene interrogata. Ne derivano due pipeline distinte, messe a confronto nella Tabella 1.4 e illustrate nella Figura 1.1: la pipeline di **ingestione**, eseguita offline, e la pipeline di **inferenza**, attivata in tempo reale a ogni richiesta utente.

Tabella 1.4: Le due pipeline del sistema RAG a confronto.

Fase	Ingestion pipeline (offline)	Query pipeline (real-time)
1	Pre-processing e chunking dei documenti sorgente	Embedding della query utente nello stesso spazio vettoriale dei documenti
2	Generazione di embedding densi e sparsi tramite modelli locali	Ricerca ibrida nel database vettoriale (<i>top-k</i>) con filtri su metadati
3	Indicizzazione in Qdrant con payload di metadati strutturati	Augmented generation: contesto recuperato + query → LLM
<i>Trigger</i>	Esecuzione manuale o schedulata (es. giornaliera)	Ad ogni richiesta utente
<i>Latenza</i>	Minuti / ore (elaborazione batch)	Millisecondi (inferenza real-time)

Pipeline di ingestione

La pipeline di ingestione trasforma i documenti grezzi in una rappresentazione adatta alla ricerca semantica. Il risultato non è più soltanto un archivio di testi, ma un indice vettoriale interrogabile per similarità. Le fasi principali sono le seguenti:

- **Pre-processing e chunking.** Il documento viene suddiviso in unità testuali (*chunk*) di dimensione controllata. La strategia di chunking influisce direttamente sulla qualità del retrieval: chunk troppo piccoli perdono contesto, mentre chunk troppo grandi possono diluire il segnale semantico e superare la finestra di contesto del modello di embedding [8].

- **Generazione degli embedding.** Ogni chunk viene trasformato in due rappresentazioni complementari. La prima è un vettore denso prodotto da un modello di embedding $f: \mathcal{T} \rightarrow \mathbb{R}^d$, in modo che documenti semanticamente simili producano vettori vicini secondo la similarità coseno:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \quad (1.3)$$

La seconda è una rappresentazione sparsa, nella quale il testo è descritto da pesi associati a termini o feature lessicali. Questa componente preserva segnali che l'embedding denso può attenuare, ad esempio nomi propri, tecnologie, sigle o termini specifici del dominio.

- **Indicizzazione nel database vettoriale.** Le rappresentazioni dense e sparse vengono persistite in **Qdrant** [55], insieme ai metadati strutturati necessari al filtraggio. Per la componente densa Qdrant usa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [23] per la ricerca approssimata (*Approximate Nearest Neighbor*, ANN); la componente sparsa abilita il matching lessicale pesato. La scelta di Qdrant rispetto ad alternative quali pgvector, Chroma e Milvus è motivata nel Capitolo 2.

Pipeline di inferenza

La pipeline di inferenza utilizza l'indice costruito offline per rispondere a una query utente. A differenza dell'ingestione, questa fase è vincolata dalla latenza percepita e deve quindi bilanciare qualità del recupero, numero di documenti restituiti e costo della generazione.

- **Embedding della query.** La query dell'utente viene trasformata nelle stesse due rappresentazioni usate in indicizzazione: un embedding denso per la similarità semantica e una rappresentazione sparsa per il matching lessicale.
- **Retrieval.** Si esegue una ricerca ibrida in Qdrant combinando il punteggio denso e quello sparso. La ricerca può essere limitata tramite *metadata filtering* a sottoinsiemi coerenti, ad esempio documenti appartenenti a una specifica epoca temporale.
- **Augmented generation.** Il contesto recuperato viene concatenato alla query nel prompt inviato all'LLM. Il modello genera la risposta basandosi sulla propria conoscenza parametrica e sulle informazioni esterne fornite.

1.3.3 Modelli di embedding

La qualità del retrieval dipende in larga misura dal modello di embedding: se la rappresentazione vettoriale non preserva correttamente il significato dei documenti, anche il modello generativo riceverà un contesto poco pertinente. Il vincolo di esecuzione on-premise restringe quindi la scelta a modelli distribuibili localmente. La versione aggiornata del progetto usa FastEmbed tramite Haystack e produce due segnali complementari: embedding densi, adatti alla similarità semantica, ed embedding sparsi, utili per il matching di termini specifici. L'analisi completa della configurazione adottata è presentata nel Capitolo 2.

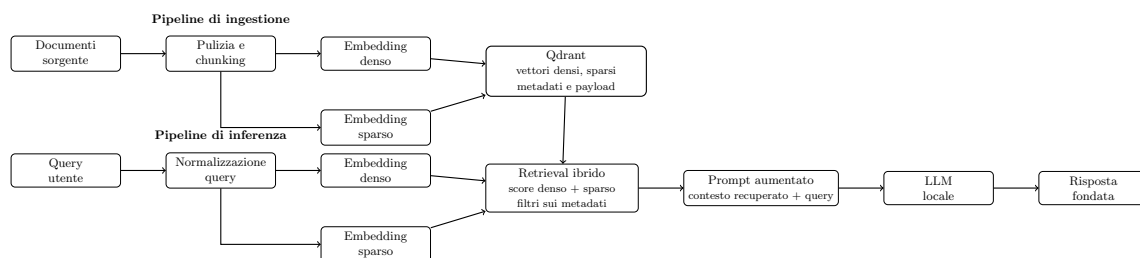


Figura 1.1: Schema della pipeline RAG ibrida: la fase di ingestione costruisce rappresentazioni dense, sparse e metadati; la fase di inferenza combina retrieval semantico, matching lessicale e filtri strutturati prima della generazione aumentata [21].

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere a query di tipo *k-nearest neighbor* (kNN). A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si cercano i vettori più vicini accettando un trade-off tra richiamo (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello nello spazio vettoriale, con complessità di ricerca approssimativamente $O(\log n)$ rispetto al numero di punti n — contro $O(n)$ della ricerca lineare esatta [23] — garantendo scalabilità anche per ampi *corpora*.

1.3.5 Agentic RAG

Il RAG lineare descritto finora segue un flusso deterministico: recupero dei documenti, costruzione del prompt e generazione della risposta. Nei casi in cui la query richiede più passaggi logici, confronto tra fonti o riformulazioni successive, tale schema può risultare insufficiente. L'*Agentic RAG* estende questo approccio utilizzando l'LLM come unità di ragionamento capace di invocare strumenti, pianificare ricerche successive e valutare la pertinenza dei documenti recuperati. Architetture come ReAct [32] formalizzano questo ciclo combinando ragionamento verbale e azioni.

In contesti RAG avanzati, tecniche come Self-RAG [2] e Corrective RAG (CRAG) [31] introducono meccanismi di controllo sulla qualità del retrieval e della risposta generata. Il sistema può così rilevare lacune informative, richiedere nuovi documenti o modificare il prompt prima di produrre la risposta finale. Le rassegne più recenti evidenziano che questa evoluzione è particolarmente utile quando le query richiedono sintesi da fonti eterogenee o passaggi intermedi di ragionamento [13].

Un'altra direzione di sviluppo è rappresentata da **GraphRAG**, formalizzato da Microsoft Research nel 2024 [10]. A differenza del RAG vettoriale classico, che è efficace nel recupero di passaggi localmente simili alla query ma meno adatto a interrogazioni globali sull'intero dataset, GraphRAG estrae entità e relazioni dai testi sorgente e costruisce un grafo di conoscenza gerarchico. In fase di retrieval, il sistema può quindi aggregare riassunti semantici di interesse *community* di concetti, ottenendo un livello di astrazione più adeguato all'analisi di corpora documentali estesi.

1.3.6 Tecniche di adattamento al dominio

Il collegamento tra LLM e conoscenza aziendale può essere realizzato in modi diversi. RAG mantiene separati i pesi del modello e il corpus documentale; il fine-tuning, invece, modifica il modello affinché si adatti a un dominio o a uno stile di risposta specifico. Questa sezione introduce le principali tecniche di adattamento a parametri ridotti, utili per inquadrare una scelta progettuale del lavoro: mantenere il corpus aziendale esterno ai pesi del modello e aggiornabile tramite pipeline di ingestione.

Fine-tuning con adattatori: LoRA

Il fine-tuning completo di un LLM richiede risorse computazionali elevate. Hu et al. [15] hanno proposto *Low-Rank Adaptation* (LoRA), che vincola gli aggiornamenti dei pesi a matrici di rango basso. Dato un layer lineare con matrice $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \quad (1.4)$$

con $\mathbf{A} \in \mathbb{R}^{r \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times r}$ e $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati; solo $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Con $r = 8$ e $m = n = 4096$, il numero di parametri addestrabili si riduce di circa due ordini di grandezza rispetto al fine-tuning completo.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA) [9] combina la quantizzazione a 4-bit del modello base con adattatori LoRA a precisione piena. I pesi vengono quantizzati in formato NF4 (*NormalFloat4*), ottimizzato per le distribuzioni tipiche dei pesi neurali; durante il forward pass i pesi vengono de-quantizzati dinamicamente in BF16, mentre gli adattatori operano in BF16.

La Tabella 1.5 riassume i principali trade-off tra le due varianti.

Tabella 1.5: Confronto tra LoRA e QLoRA per un modello base da 7B parametri.

Caratteristica	LoRA	QLoRA
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead de-quantizzazione	assente	presente
Degradazione qualitativa stimata	trascurabile	<1% su benchmark principali

DoRA: Weight-Decomposed Low-Rank Adaptation

Nel 2024 Liu et al. hanno introdotto **DoRA** (*Weight-Decomposed Low-Rank Adaptation*) [22]. Il metodo parte dall'osservazione che LoRA approssima bene l'aggiornamento direzionale dei pesi, ma cattura con minore efficacia le variazioni di magnitudine osservate nel fine-tuning completo. DoRA separa quindi il tensore dei pesi in una componente di magnitudine e una componente direzionale: la prima viene aggiornata direttamente, mentre la seconda sfrutta l'approssimazione a rango basso di LoRA. In questo modo si riduce il divario prestazionale rispetto al fine-tuning completo, mantenendo un numero contenuto di parametri addestrabili.

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input senza modificare i pesi. Tecniche come *few-shot prompting* [6] e *chain-of-thought prompting* [30] guidano il modello verso risposte più strutturate, arricchendo il prompt con esempi o istruzioni di ragionamento. Nel sistema descritto in questa tesi, il prompt engineering è impiegato nella definizione del *system prompt* e nell'iniezione strutturata del contesto RAG. Le strategie adottate sono descritte nel Capitolo 3, insieme agli endpoint che costruiscono il prompt aumentato.

1.4 Framework di orchestrazione: Haystack

La costruzione di un sistema RAG non consiste soltanto nella scelta del modello generativo. È necessario coordinare modelli di embedding, database vettoriali, LLM e fasi di pre-processing attraverso interfacce coerenti e componibili. **Haystack** [40], sviluppato da deepset, è un framework Python open source progettato per organizzare questi componenti in pipeline esplicite.

Nella versione 2.x adottata in questo progetto, ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, collegabili in modo dichiarativo. Sono disponibili integrazioni native con Qdrant (`QdrantDocumentStore`), FastEmbed (`FastembedDocumentEmbedder`, `FastembedTextEmbedder` e componenti sparse corrispondenti), Ollama (`OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`). Il dettaglio implementativo della pipeline di ingestione e di retrieval è descritto nel Capitolo 3.

1.5 Sintesi e posizionamento del lavoro

Il presente capitolo ha fornito il quadro tecnico necessario a comprendere il progetto aziendale descritto nei capitoli successivi. Le architetture Transformer costituiscono il substrato comune a generazione ed embedding; i loro limiti in contesti a conoscenza chiusa, cioè conoscenza statica, allucinazioni e assenza di accesso nativo ai dati interni, motivano l'adozione del paradigma RAG. La scelta di operare con modelli e infrastrutture on-premise, come Ollama, Qdrant e Haystack, risponde ai vincoli di riservatezza e manutenibilità del caso i3Guild. Il capitolo successivo traduce questi fondamenti in requisiti, scelte tecnologiche e architettura del sistema.

Capitolo 2

Analisi dei Requisiti e Progettazione del Sistema

Il capitolo traduce il quadro teorico introdotto in precedenza in un problema progettuale concreto. L'obiettivo non è costruire un sistema RAG generico, ma integrare un assistente nel contesto operativo di Intré, rispettando i vincoli organizzativi, infrastrutturali e di riservatezza già presenti. Per questo la trattazione parte dal modello aziendale delle Gilde, identifica il caso d'uso e il corpus informativo disponibile, formalizza i requisiti e motiva infine le scelte tecnologiche che portano alla proposta architettuale.

2.1 Il contesto aziendale: Intré S.r.l.

Intré S.r.l.¹ è una software factory italiana con sede a Monza. L'azienda sviluppa prodotti digitali per settori eterogenei, tra cui energia, assicurazioni, automotive ed entertainment, e organizza il lavoro in team Scrum cross-funzionali. La dimensione aziendale, pari a circa settanta persone, consente di mantenere processi interni formalizzati senza rinunciare a una circolazione diretta della conoscenza tra i team [16].

Il payoff *Learn / Code / Deploy Value* esplicita l'ordine con cui Intré interpreta il proprio modo di produrre valore: prima l'apprendimento, poi la realizzazione del software, infine il rilascio di valore al cliente. Questa impostazione è coerente con il concetto di *Learning Organization* proposto da Peter Senge in *The Fifth Discipline* [25]. Nel modello di Senge l'apprendimento organizzativo non coincide con la sola formazione individuale, ma richiede padronanza personale, modelli mentali esplicitati, visione condivisa, apprendimento di gruppo e pensiero sistemico. La quinta disciplina, cioè il pensiero sistemico, collega le altre quattro e permette di leggere l'organizzazione come un insieme di relazioni, feedback e dipendenze, non come una somma di attività isolate.

Nel caso di Intré questa prospettiva non rimane un principio astratto. Il modello aziendale prevede un insieme di pratiche dedicate all'apprendimento continuo: Gilde, Camp, Community, Skill Matrix, certificazioni, Academy, Community of Practice per speaker, obiettivi di apprendimento, formazione in aula e corsi di lingua [16]. Tali pratiche rendono l'apprendimento una responsabilità distribuita, sostenuta da tempo, budget e momenti di restituzione pubblica. La certificazione

¹<https://www.intre.it>

ISO 27001 e le competenze maturate in ambiti come cloud-native, cybersecurity e intelligenza artificiale rendono inoltre particolarmente rilevante il tema della gestione sicura della conoscenza interna.

2.1.1 Il modello delle Gilde

Le **Gilde** (*gilde*) sono uno degli strumenti principali con cui Intré rende operativa la propria idea di apprendimento organizzativo. Sono cicli rapidi di apprendimento di gruppo, della durata di quattro mesi, costruiti a partire da temi proposti direttamente dalle persone dell'azienda [16, 17]. Una Gilda nasce quando una proposta raccoglie un numero minimo di partecipanti; il gruppo lavora in autogestione e dedica al tema scelto una quota stabile del proprio tempo lavorativo. L'esito non è un esame o una valutazione formale, ma un risultato di apprendimento da presentare al Camp successivo. Il modello combina quindi autonomia nella scelta dei temi, durata definita, lavoro di gruppo e produzione di un risultato condivisibile. Le sue caratteristiche principali sono:

- **Formazione spontanea:** ogni persona può proporre o aderire a una Gilda su qualsiasi argomento, tecnico o non tecnico (es. linguaggi di programmazione, ceramica, game development, intelligenza artificiale).
- **Ciclo temporale (Epoch):** le Gilde si ricreano ogni quattro mesi; ogni ciclo costituisce un'“epoca” distinta, con nuovi temi e nuovi gruppi. Questa scansione periodica evita che l'apprendimento rimanga legato a iniziative isolate e lo inserisce in una routine aziendale riconoscibile.
- **Output condivisibile:** al termine del periodo la Gilda produce un risultato tangibile (prototipo, report, repository, presentazione) che viene archiviato e condiviso. La restituzione pubblica durante il Camp trasforma l'esperienza del gruppo in conoscenza disponibile anche per il resto dell'organizzazione.
- **Ampiezza tematica:** dall'archivio pubblico del sito aziendale emergono Gilde su temi come *Angular*, *F#*, *IA agentiva*, *game development* con Phaser, ceramica, selezione fotografica con AI, e molti altri [17].

Il ciclo di vita delle Gilde è gestito tramite **i3Guild**, una piattaforma web interna sviluppata con Next.js, PostgreSQL e Keycloak. Questa applicazione conserva nel tempo proposte, descrizioni, partecipanti e risultati delle Gilde. Di conseguenza, i3Guild non è soltanto uno strumento amministrativo: è una memoria organizzativa parziale, nella quale si depositano interessi, competenze, esperimenti e risultati prodotti nel tempo. Proprio per questo rappresenta sia il contesto applicativo di integrazione sia la principale sorgente dati del sistema RAG progettato in questa tesi.

2.2 Caso d'uso: Proposta di nuove Gilde Intré

Il caso d'uso individuato riguarda il supporto alla proposta di nuove Gilde. Quando una persona vuole avviare una nuova iniziativa, può essere utile conoscere esperienze precedenti, temi già affrontati, risultati ottenuti e partecipanti coinvolti. Oggi queste informazioni esistono, ma sono distribuite tra database e contenuti editoriali. Il sistema proposto mira a renderle consultabili tramite query in linguaggio naturale, generando risposte contestualizzate e ancorate a fonti interne. A tale

scopo viene adottato il paradigma *Retrieval-Augmented Generation* (RAG), introdotto da Lewis et al. [20] per estendere la memoria non parametrica dei Large Language Model (LLM) tramite recupero dinamico di documenti da un corpus esterno.

2.2.1 Motivazione del caso d'uso

Il corpus delle Gilde archiviate in i3Guild, insieme ai relativi blog post, costituisce una base di conoscenza aziendale di valore: raccoglie descrizioni di progetti, obiettivi, rischi, risultati raggiunti e annotazioni dei partecipanti, accumulate in oltre trenta epoch a partire dal 2016 [17]. Tuttavia questa conoscenza è frammentata. Una parte risiede in un database relazionale, ottimo come sorgente applicativa ma poco accessibile tramite domande in linguaggio naturale; un'altra parte è distribuita in post e contenuti non sempre facili da reperire. Un assistente RAG consente di trasformare tale patrimonio in uno strumento operativo: riduce il tempo di ricerca, facilita il riuso delle esperienze pregresse e rende più trasferibile la conoscenza tra i team.

2.2.2 Obiettivi del caso d'uso

- Fornire risposte affidabili e tracciabili su contenuti presenti nei repository aziendali, riducendo il rischio di allucinazioni tipico della generazione puramente parametrica [20].
- Ridurre il tempo di ricerca delle informazioni per attività operative e decisionali interne a Intré.
- Consentire l'esecuzione del servizio in ambiente on-premise per requisiti di sovranità del dato, in conformità al Regolamento Generale sulla Protezione dei Dati (GDPR) [12].
- Integrare il sistema nel flusso i3Guild esistente, minimizzando l'impatto sul processo operativo consolidato.

2.2.3 Descrizione dei dati e analisi del corpus

Per progettare correttamente la pipeline di ingestione è necessario chiarire quali informazioni debbano alimentare l'indice. I dati di ingresso al sistema RAG comprendono tre categorie:

- **Documenti delle Gilde:** specifiche di progetto, obiettivi, rischi, risultati attesi e raggiunti, annotati con metadati strutturati (epoche, partecipanti, modalità di partecipazione).
- **Blog post:** post pubblicati dai partecipanti alle gilde che ne sintetizzano i risultati e le esperienze maturate.
- **Documenti non strutturati:** file PDF, HTML e note operative con metadati associati.

Dall'analisi del corpus i3Guild, estratto il 15/02/2026, emergono le statistiche riportate nelle Tabelle 2.1 e 2.2. Questi valori non hanno solo funzione descrittiva: determinano la granularità dell'indicizzazione, la strategia di chunking e la scelta del modello di embedding.

La somma media dei campi testuali rilevanti (`summary + goals + outcome + risks + achievedResultsDescription`) ammonta a circa 1.900 caratteri, corrispondenti a circa 400 token. Anche nel caso peggiore la somma non supera i 7.000–8.000 token, rimanendo compatibile con la finestra di contesto del modello di embedding selezionato (Sezione 2.4.3). Da ciò deriva una conseguenza progettuale importante:

Tabella 2.1: Statistiche delle entità principali del corpus i3Guild (estrazione 15/02/2026).

Entità	Conteggio	Note
Gilda	270	Gestibile per re-indicizzazioni complete
Epoch	30	≈ 9 gilde per epoca in media
Relazioni Utente-Gilda	1.166	$\approx 4,3$ partecipanti medi per gilda
Utenti esterni	3	Trascurabile ai fini del RAG

Tabella 2.2: Statistiche dei campi testuali delle entità Gilda (valori in caratteri).

Campo	Media	Min	Max	Vuoti	Note
summary	610	0	18.224	5%	Campo più ricco; outlier rilevante
goals	258	0	2.698	35%	Spesso breve o assente
outcome	133	0	2.145	36%	Spesso breve o assente
risks	106	0	1.245	38%	Spesso breve o assente
achievedResultsDescription	782	0	6.421	74%	Lungo quando presente
journal	≈ 1	1	1	99%	Praticamente inutilizzato; escluso

nella maggior parte dei casi una Gilda può essere trattata come documento consolidato, senza frammentare aggressivamente i campi testuali e senza perdere il contesto semantico che lega obiettivi, rischi e risultati.

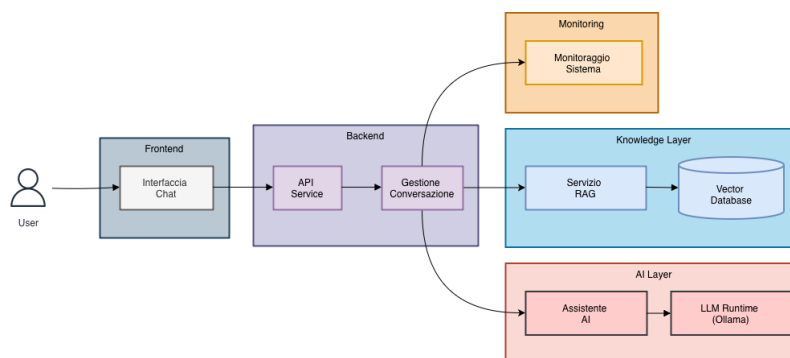


Figura 2.1: Diagramma del caso d'uso “Intré”: attori, interazioni principali e confini del sistema.

2.3 Requisiti di sistema

Il caso d'uso appena descritto permette di passare dalla motivazione generale ai requisiti del sistema. Essi sono stati ricavati dall'analisi del corpus, dai vincoli infrastrutturali di Intré e dalle policy aziendali in materia di protezione dei dati. La Tabella 2.3 sintetizza i requisiti ad alto livello, suddivisi per categoria.

Tabella 2.3: Riepilogo requisiti ad alto livello.

Categoria	Requisiti principali
Funzionali	Ricerca semantica full-text su documenti interni; generazione di risposte contestualizzate con riferimenti alle fonti; capacità di filtraggio per metadati (epoca, modalità di partecipazione, proprietario della gilda).
Privacy e conformità	Esecuzione on-premise; nessun invio di dati a servizi cloud esterni; logging e auditing degli accessi; protezione dei dati in conformità al GDPR [12]. I rischi relativi alla privacy nei sistemi RAG, tra cui la possibile estrazione dei dati di retrieval, sono discussi in letteratura [33]; la scelta on-premise costituisce la principale misura mitigativa adottata.
Performance	Latenza compatibile con uso interattivo per query semplici, distinguendo tra modello già caricato e cold start; throughput adeguato a carichi concorrenti moderati; inferenza CPU-only come scenario primario.
Costi e sostenibilità	Minimizzazione dei costi operativi tramite uso efficiente delle risorse (quantizzazione dei modelli, inferenza CPU); adozione di soluzioni open-source senza costi per licenze.
Manutenibilità	Componenti modulari con interfacce standardizzate; automazione della pipeline di ingestione; documentazione e test automatizzati.

2.3.1 Vincoli architetturali

I requisiti precedenti si traducono in alcuni vincoli architetturali non negoziabili, che orientano direttamente la scelta dei componenti:

- **On-premise obbligatorio:** i dati delle Gilde, incluse informazioni sui partecipanti e sui progetti, non possono transitare su infrastrutture cloud di terze parti. Il deployment locale consente di soddisfare il principio di integrità e riservatezza previsto dal GDPR [12].
- **Compatibilità con lo stack i3Guild:** l'integrazione deve essere coerente con l'infrastruttura esistente (Docker Compose, PostgreSQL/Prisma, Next.js, Keycloak per l'autenticazione).
- **Limitazioni hardware:** l'ambiente Docker dispone di 8 GB di RAM complessivi, condivisi tra database, servizi applicativi, pipeline di embedding quando attiva e modello di generazione del testo. Questo vincolo è determinante per la selezione del modello LLM (Sezione 2.4.4).
- **No API esterne per LLM ed embedding:** generazione ed embedding devono restare locali; Ollama gestisce il modello generativo, mentre FastEmbed/Haystack gestisce i modelli di embedding senza dipendere da API di provider esterni.

2.4 Scelte tecnologiche: analisi comparativa

Le scelte tecnologiche sono state effettuate a partire dai vincoli appena definiti. Per ciascun componente principale dello stack sono state valutate alternative compatibili con l'esecuzione on-premise,

privilegiando soluzioni semplici da integrare nello stack i3Guild, osservabili e sostenibili rispetto alle risorse disponibili.

2.4.1 Framework di orchestrazione RAG

Il framework di orchestrazione deve collegare ingestione, embedding, retrieval e generazione senza introdurre eccessiva complessità operativa. Sono stati considerati tre framework open-source: **LangChain**, **LlamaIndex** e **Haystack 2.x**.

LangChain offre l’ecosistema più ampio di integrazioni e una forte vocazione verso workflow agentici e *multi-step reasoning*. Nel 2024 l’introduzione di **LangGraph** [45] ha ampliato questa impostazione, permettendo di modellare applicazioni multi-agente come grafi di stato. Tale flessibilità è utile in scenari complessi, ma nel progetto in esame introduce un costo non trascurabile in termini di curva di apprendimento, stabilità delle API e manutenzione, anche alla luce dei *breaking changes* frequenti riportati in letteratura [52].

LlamaIndex è orientato alla gestione dei dati e al retrieval avanzato. Offre numerosi data connector e strutture di indicizzazione specializzate, risultando particolarmente adatto a pipeline di question-answering documentale [36]. Rispetto al caso considerato, tuttavia, l’obiettivo principale non è solo indicizzare sorgenti eterogenee, ma integrare una pipeline osservabile e facilmente testabile nello stack applicativo esistente.

Haystack 2.x (deepset) adotta un modello a componenti tipizzati con input/output espliciti, organizzati in pipeline dichiarative serializzabili in YAML. Studi comparativi evidenziano che Haystack offre il minore overhead di orchestrazione (≈ 5.9 ms) e il minore consumo di token rispetto a LangChain (≈ 10 ms) su benchmark standardizzati [34]. La sua architettura pipeline-first favorisce la tracciabilità, i test e la sostituzione dei componenti in produzione [51].

Tabella 2.4: Confronto sintetico dei framework RAG candidati.

Criterio		LangChain	LlamaIndex	Haystack 2.x
Filosofia	orche- strazione	Agente/imperativa	Data-first	Pipeline dichiarativa
Overhead		≈ 10 ms	≈ 6 ms	≈ 5.9 ms
Integrazione		Nativa	Nativa	Nativa
Qdrant	Olla- ma	Sì	Sì	Sì
Integrazione		Sì	Sì	Sì
Testabilità pipeline		Media	Media	Alta
Serializzazione	YAML	No	Parziale	Sì
Adatto per produ- zione RAG		Sì (con overhead)	Sì	Sì (preferito)

Scelta motivata: Haystack 2.x. L’architettura a componenti tipizzati, la documentazione rigorosa, le integrazioni native con Qdrant (`QdrantDocumentStore`), FastEmbed per embedding densi

e sparsi e Ollama per la generazione, il minore overhead di orchestrazione e la serializzabilità delle pipeline rendono Haystack la scelta più adeguata per un sistema RAG production-oriented in ambiente on-premise [52].

2.4.2 Database vettoriale

La ricerca di similarità approssimata (*Approximate Nearest Neighbor*, ANN) su vettori ad alta dimensionalità è il componente infrastrutturale che abilita il retrieval. La scelta del database vettoriale incide su latenza, filtraggio per metadati, semplicità di deployment e capacità di manutenzione dell'indice. Sono stati quindi valutati quattro database vettoriali self-hostable.

Tabella 2.5: Analisi comparativa dei database vettoriali candidati.

Database	Impl.	Indice	Filtering	Dashboard	Deployment
Qdrant	Rust	HNSW	Pre-filtering payload	Web UI integrata	1 container
pgvector	C	IVFFlat / HNSW	SQL WHERE	No (via PgAdmin)	Estensione PG
Milvus	Go/C++	HNSW, IVF	Attribute-based	Attu (separato)	Multi-container
Chroma	Python	HNSW (hnswlib)	Metadata base	No	1 container

Il confronto non si limita alle prestazioni pure: nel contesto i3Guild hanno peso anche la semplicità operativa e la possibilità di applicare filtri sui metadati delle Gilde. I principali criteri di esclusione e selezione sono i seguenti:

- **pgvector** è stato considerato in quanto l'infrastruttura i3Guild utilizza già PostgreSQL. Tuttavia, la letteratura segnala che le prestazioni di pgvector degradano significativamente oltre le centinaia di migliaia di vettori, con ritardi fino a 15 volte superiori rispetto a Qdrant in termini di throughput su dataset grandi [44]. Benchmark più recenti mostrano un divario più contenuto (Qdrant: ≈ 41 QPS a 99% recall su 50M vettori; pgvectorscale: ≈ 471 QPS), ma evidenziano che Qdrant ottiene latenze tail significativamente migliori (39% migliore a p95, 48% migliore a p99) [59]. Assenza di dashboard integrata e di payload filtering flessibile completano le ragioni per la non-selezione.
- **Milvus** è stato escluso per l'elevata complessità di deploy: richiede componenti aggiuntivi (etcd, MinIO), aumentando il carico operativo incompatibile con i vincoli del progetto.
- **Chroma** è implementato in Python e soffre di instabilità della persistenza dati nelle versioni precedenti; è più adatto a prototipi che a sistemi di produzione.
- **Qdrant** è scritto in Rust e offre buone prestazioni con latenze stabili nei benchmark disponibili. La funzionalità di pre-filtering, che applica i filtri sui payload *prima* della ricerca vettoriale, è particolarmente rilevante per query contestuali come “gilda dell'epoca X”: il filtering riduce lo spazio di ricerca prima del calcolo ANN, mantenendo il top- K accurato. Offre Web UI integrata su porta 6333, client TypeScript ufficiale compatibile con Next.js, e deploy in singolo container Docker [56].

Scelta motivata: Qdrant. Per le ragioni sopra esposte e in linea con il vincolo aziendale di self-hosting, Qdrant rappresenta il database vettoriale più adeguato al contesto del progetto.

2.4.3 Modello di embedding

Il modello di embedding converte i documenti testuali in rappresentazioni numeriche interrogabili da Qdrant. Nella prima versione del progetto questa funzione era affidata a Ollama e a un singolo vettore denso; la versione aggiornata separa invece l’inferenza generativa dall’indicizzazione e usa FastEmbed tramite Haystack. La pipeline produce sia embedding densi sia embedding sparsi, così il retrieval può combinare similarità semantica e corrispondenza lessicale su nomi propri, tecnologie e termini ricorrenti nelle Gilde [37, 38].

Tabella 2.6: Configurazione embedding della versione aggiornata del sistema RAG.

Ruolo	Componente Haystack	Default	Funzione
Embedding denso	FastembedDocumentEmbedder / FastembedTextEmbedder	BAAI/bge-base-en-v1.5	Similarità semantica tra query e documenti.
Embedding sparso	FastembedSparseDocumentEmbedder / FastembedSparseTextEmbedder	openai/ada-002 / Splade-Ranking	Matching lessicale pesato, utile per termini specifici.
Store ibrido	QdrantDocumentStore con use_sparse_embeddings=true	i3guild_knowledge	Persistenza con vettori densi e rappresentazioni sparse.

La scelta aggiornata risponde a tre esigenze. La prima è eliminare la dipendenza da Ollama nella fase batch di indicizzazione: Ollama resta dedicato alla generazione LLM, mentre FastEmbed produce gli embedding in un job separato. La seconda è rendere il retrieval più robusto rispetto alle sole similarità dense. La terza è avvicinare ingestione e query allo stesso schema operativo: in scrittura vengono generati embedding densi e sparsi dei documenti; in lettura la query viene trasformata nelle due rappresentazioni corrispondenti e inviata a QdrantHybridRetriever [39]. Il valore `EMBEDDING_DIMENSIONS=768` rimane il riferimento per la componente densa.

La scelta della ricerca ibrida non deriva da una valutazione quantitativa condotta sul corpus i3Guild, perché durante il progetto non è stato costruito un dataset annotato con query e documenti rilevanti. È però coerente con evidenze esterne recenti: nei task RAG di TREC 2025 un sistema sparse+dense con reranking ottiene i migliori risultati riportati dal team NITATREC (*MAP* 0.1037, *nDCG@30* 0.527 e *Recall@100* 0.158) [27]; in un benchmark finanziario su documenti misti testo-tabella, una pipeline a due stadi con retrieval ibrido e reranking raggiunge *Recall@5* 0.816 e *MRR@3* 0.605, superando i metodi single-stage [1]. Altri studi mostrano però che il vantaggio non è universale: nel dominio biomedicale una baseline densa resta molto competitiva e le differenze tra strategie dipendono dal corpus e dalle metriche considerate [4]. Per i3Guild questi risultati vanno quindi usati come supporto alla scelta progettuale, non come prova sperimentale locale: la componente sparsa è motivata soprattutto dalla presenza di nomi di Gilde, ruoli, modalità di partecipazione e termini tecnici che hanno valore informativo letterale.

2.4.4 Server di inferenza LLM e selezione del modello di chat

Il componente di generazione del testo è gestito da **Ollama**, server di inferenza locale che supporta modelli in formato GGUF e consente l’uso di varianti quantizzate a 4 e 8 bit. Nel prototipo aziendale

la scelta del modello di chat è stata condizionata da un vincolo operativo preciso: l'intero stack Docker dispone di risorse limitate e deve eseguire, nello stesso ambiente, PostgreSQL, Qdrant, il servizio Next.js, la pipeline RAG e Ollama. Per questo la selezione non mira a identificare il modello migliore in senso assoluto, ma un modello sufficientemente stabile per validare l'integrazione end-to-end.

Scelta motivata: gemma3:1b. Il modello adottato nel prototipo è **gemma3:1b**. La scelta deriva da prove preliminari interne su modelli disponibili nel registry Ollama, valutati rispetto a tre criteri: latenza percepita durante l'interazione, consumo di memoria compatibile con gli altri container e qualità minima delle risposte in presenza di contesto RAG. Le motivazioni principali sono:

- **RAM compatibile con i vincoli:** il modello può essere caricato senza saturare le risorse condivise con PostgreSQL, Qdrant, Next.js e il servizio RAG.
- **Latenza accettabile:** i tempi di risposta sono compatibili con un uso interattivo del chatbot, tenendo conto del cold start del modello.
- **Qualità sufficiente per il prototipo:** le risposte risultano adeguate quando il modello riceve contesto recuperato da Qdrant.
- **Integrazione semplice:** il modello è disponibile direttamente in Ollama e non richiede conversioni o runtime aggiuntivi.

Questa valutazione resta volutamente interna al progetto aziendale: serve a rendere eseguibile il prototipo i3Guild, non a produrre conclusioni generali sull'inferenza locale. La generalizzazione del problema, cioè la misura sistematica di memoria, cold start, *time-to-first-token*, throughput e impatto della quantizzazione su Apple Silicon, viene affrontata nei Capitoli ?? e ??.

2.5 Proposta architetturale

Le scelte tecnologiche descritte convergono in una soluzione modulare e containerizzata. L'architettura separa responsabilità diverse – ingestione e normalizzazione dei dati, generazione degli embedding, persistenza vettoriale, retrieval e inferenza LLM – in modo da mantenere il sistema osservabile, sostituibile e coerente con lo stack i3Guild. La Figura 2.2 riporta lo schema di alto livello dell'architettura.

2.5.1 Componenti principali

I componenti non sono indipendenti solo dal punto di vista del deployment: ciascuno risponde a un requisito emerso nelle sezioni precedenti. PostgreSQL rimane la sorgente autoritativa, Qdrant rende interrogabile il corpus per similarità, Ollama evita dipendenze cloud e FastAPI espone un confine applicativo semplice verso Next.js.

Ingestione e pre-processing (i3guild-embedding-pipeline) Pipeline batch Python che esegue le fasi ETL descritte nella Sezione 2.5.3. Acquisisce i documenti da PostgreSQL, normalizza il testo, esegue il chunking, genera embedding densi e sparsi con FastEmbed/Haystack e indicizza su Qdrant.

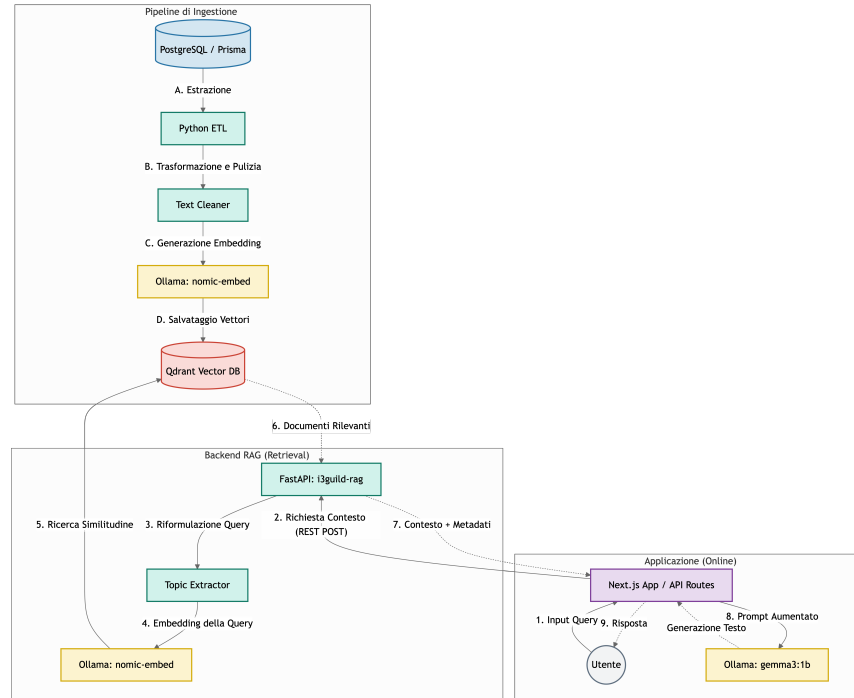


Figura 2.2: Schema architetturale del sistema RAG on-premise per i3Guild. Il flusso di ingestione (ETL) popola Qdrant a partire da PostgreSQL; il flusso di retrieval serve le query di Next.js tramite FastAPI.

Strategia di chunking Sulla base dell'analisi dei dati (Sezione 2.2.3), viene adottato l'approccio a *documento consolidato* (1 Gilda $\leftrightarrow$ 1 documento vettoriale): tutti i campi testuali rilevanti sono concatenati in un unico documento con separatori strutturati. Per gli outlier che superano il limite configurato viene applicato chunking con overlap, mantenendo nel chunk l'intestazione della Gilda. Questo approccio preserva il contesto semantico nella maggior parte dei casi e mantiene gestibili upsert e cancellazioni su Qdrant.

Database vettoriale (Qdrant) Collection principale `i3guild_knowledge` configurata per retrieval ibrido: vettore denso a 768 dimensioni, rappresentazione sparsa e payload JSON con metadati indicizzati per il filtering (Sezione 2.5.2).

Orchestrazione RAG (Haystack 2.x) Haystack è usato sia nel job di indicizzazione sia nel servizio di query. La pipeline batch collega `FastembedDocumentEmbedder`, `FastembedSparseDocumentEmbedder` e `DocumentWriter`; la pipeline di query collega `FastembedTextEmbedder`, `FastembedSparseTextEmbedder` e `QdrantHybridRetriever`.

Inference LLM (Ollama) Server di inferenza locale; modello di chat adottato: `gemma3:1b`, selezionato sulla base di prove preliminari coerenti con i vincoli del prototipo (Sezione 2.4.4).

Servizio API (FastAPI) Microservizio Python che espone endpoint di retrieval e generazione, tra cui `POST /rag/retrieve` (porta 8000, mappata a 8002 su host). Restituisce documenti con score di similarità e campi diagnostici.

Osservabilità e sicurezza Logging centralizzato, metriche di latenza e utilizzo risorse; autenticazione tramite Keycloak (già presente in i3Guild); cifratura a riposo per i dataset sensibili.

2.5.2 Progettazione della collection Qdrant

La collection `i3guild_knowledge` è il punto in cui la conoscenza estratta da i3Guild viene trasformata in indice interrogabile. La configurazione, riportata nella Tabella 2.7, supporta retrieval ibrido: similarità vettoriale densa e componente sparsa nello stesso store.

Tabella 2.7: Parametri di configurazione della collection Qdrant.

Parametro	Valore	Motivazione
<code>embedding_dim</code>	768	Dimensione della componente densa FastEmbed
<code>vectors.distance</code>	Cosine	Standard per similarità semantica testuale
<code>use_sparse_embeddings</code>	true	Abilita la componente sparsa per retrieval ibrido
<code>recreate_index</code>	true (job batch)	Ricrea collection dense-only mantenendo lo stesso nome
<code>hnsw_config.m</code>	16 (default)	Compromesso velocità/precisione
<code>hnsw_config.ef_construct</code>	100 (default)	Qualità dell'indice in fase di build

Ogni punto nella collection porta un payload JSON con i campi elencati nella Tabella 2.8. I campi marcati come indicizzati abilitano il pre-filtering, caratteristica distintiva di Qdrant che applica i filtri sul payload *prima* della ricerca vettoriale, garantendo che il calcolo del top- K sia effettuato solo sul sottoinsieme filtrato.

Tabella 2.8: Struttura del payload Qdrant per la collection `i3guild_knowledge`.

Campo	Tipo	Descrizione	Indicizzato
<code>guild_id</code>	integer	ID della gilda sorgente	Sì
<code>guild_name</code>	keyword	Nome della gilda	Sì
<code>epoch_id</code>	integer	ID dell'epoca di appartenenza	Sì
<code>epoch_start</code>	datetime	Data inizio epoca	Sì
<code>epoch_end</code>	datetime	Data fine epoca (nullable)	Sì
<code>participation_mode</code>	keyword	FullRemote / Hybrid / OnSite	Sì
<code>is_individual</code>	bool	Gilda individuale	Sì
<code>owner_id</code>	keyword	ID del proprietario	Sì
<code>participants</code>	keyword[]	Nomi dei partecipanti	Sì
<code>participant_count</code>	integer	Numero di partecipanti	No
<code>source_text</code>	text	Testo originale del chunk	No
<code>last_synced</code>	datetime	Timestamp ultima sincronizzazione	No

Gli ID dei documenti sono deterministici nel formato `guild-{guild_id}-chunk-{index}`. In questo modo la reindicizzazione della stessa Gilda produce gli stessi identificativi e consente a Qdrant di sovrascrivere i punti esistenti quando la policy è `DuplicatePolicy.OVERWRITE`.

2.5.3 Pipeline di ingestione ETL

La pipeline ETL ha il compito di mantenere allineata la rappresentazione vettoriale con i dati applicativi. Il job `i3guild-embedding-pipeline/pipeline/main.py` esegue le seguenti fasi:

1. **Extract:** connessione a PostgreSQL e query SQL con JOIN su `Epoch`, `GuildUserRelation` ed `ExternalUser` per estrarre tutti i campi semantici delle Gilde insieme ai nomi dei partecipanti.
2. **Transform:** normalizzazione HTML tramite `BeautifulSoup`, composizione di documenti Haystack con sezioni marcate e chunking con overlap quando il testo supera `INGEST_MAX_DOCUMENT_CHARS`.
3. **Embed:** generazione di embedding densi tramite `FastembedDocumentEmbedder` e di embedding sparsi tramite `FastembedSparseDocumentEmbedder`.
4. **Load:** scrittura su Qdrant tramite `DocumentWriter` con policy configurabile, di default `DuplicatePolicy.OVERWRITE`.

La struttura della pipeline Haystack è riportata in Appendice B, Listato B.1. Nel capitolo principale è sufficiente evidenziare la sequenza logica Extract–Transform–Embed–Load, perché il dettaglio del codice non modifica la scelta progettuale.

Il container della pipeline mantiene comportamento batch: esegue ingestione, upsert su Qdrant e termina con codice 0 in caso di successo o 1 in caso di errore. Questa separazione evita che indicizzazione e query interattive competano nello stesso processo.

2.5.4 Pipeline di retrieval e logica di ranking

La pipeline di retrieval usa l'indice ibrido Qdrant per recuperare i documenti più pertinenti alla query e restituisce al livello applicativo un contesto già filtrato. La query viene trasformata sia in embedding denso sia in embedding sparso; le due rappresentazioni alimentano `QdrantHybridRetriever`. Il sistema non adotta un valore `top.k` fisso: una soglia rigida rischierebbe infatti di includere documenti marginali nelle query semplici o di tagliare informazioni utili nelle query più ampie. Per questo viene applicato un approccio adattivo in tre stadi:

1. **Soglia di similarità** (default: $\text{cosine score} \geq 0.7$): esclude documenti semanticamente non pertinenti.
2. **Rilevamento del calo relativo:** se la caduta percentuale tra due score consecutivi supera una soglia configurabile (default: 0.08), la lista viene troncata al confine naturale della rilevanza.
3. **Cap di sicurezza** (default: ≤ 10 documenti): limite massimo per controllare la dimensione del contesto passato al modello generativo.

Prima del taglio finale viene applicata una deduplicazione post-retrieval: il servizio rimuove documenti ripetuti per ID, fingerprint testuale e similarità fuzzy. Una cache in-process con TTL breve riduce inoltre richieste ripetute con stessi parametri, senza cambiare il modello dati persistente.

Query rewriting e Topic-First Search

Nel contesto i3Guild, molte query contengono parole ricorrenti come “gilda”, “fondare” o “consigli”. Questi termini descrivono l’interazione con il sistema, ma non il tema informativo cercato; se mantenuti nella query, possono dominare lo spazio degli embedding e produrre risultati generici. Il *query rewriting*, discusso anche in framework come RQ-RAG [7], viene quindi utilizzato per isolare il topic rilevante prima del retrieval. L’approccio implementato tramite la funzione `extract_topic_query()` opera in tre passaggi:

1. Normalizzazione e rimozione della punteggiatura.
2. Filtraggio di un insieme di circa 80 *domain stopwords* (DOMAIN_STOPWORDS): termini specifici del dominio i3Guild e stopwords italiane generiche.
3. Estrazione del topic residuo; se diverso dalla query originale, la ricerca viene eseguita sul topic pulito (*topic-first strategy*).

La Tabella 2.9 illustra l’effetto del rewriting su query campione.

Tabella 2.9: Effetto del query rewriting su esempi di query utente.

Query originale	Topic estratto	Risultato atteso
“vorrei fondare una gilda sulla musica, cosa mi consigli?”	musica	Gilda a tema musicale
“intelligenza artificiale”	intelligenza artificiale	(nessun rewriting)
“chi ha partecipato al progetto di design?”	design	Gilda con partecipanti nel settore design

2.5.5 Endpoint API di riferimento

Il risultato della pipeline viene esposto tramite un endpoint FastAPI, che costituisce il contratto tra il servizio RAG e l’applicazione Next.js. L’endpoint principale è il seguente:

POST /rag/retrieve

Un esempio di request body e la struttura della response sono riportati in Appendice B, Listati B.2 e B.3.

I campi `total_candidates`, `threshold_used`, `cutoff_reason` e `topic_query` (presente solo quando il rewriting è attivo) sono utili per il debug, per il controllo applicativo e per eventuali verifiche successive della qualità del retrieval.

2.6 Integrazione con i3Guild

L’integrazione del sistema RAG nell’infrastruttura i3Guild esistente è pensata per ridurre l’impatto sul sistema già operativo. PostgreSQL, Next.js, Keycloak e Kong mantengono il proprio ruolo; il sistema RAG si aggiunge come insieme di servizi nel file `dev_containers/docker-compose.yml`. Questa scelta permette di introdurre il retrieval semantico senza modificare il modello dati principale né il flusso di autenticazione.

2.6.1 Servizi Docker aggiuntivi

La Tabella 2.10 elenca i container aggiuntivi introdotti dal sistema RAG.

Tabella 2.10: Servizi Docker introdotti dall'integrazione RAG in i3Guild.

Container	Immagine	Porta host	Ruolo
qdrant	qdrant:latest	6333, 6334	Database vettoriale; Web UI su /dashboard
ollama	ollama:latest	11434	Server di inferenza LLM
i3guild_rag	Build locale	8002	FastAPI + Haystack (retrieval ibrido e generazione)
i3guild_embedding_pipeline	Build locale	–	Job batch PostgreSQL → FastEmbed/Haystack → Qdrant
postgres	già presente	già presente	Sorgente autoritativa relazionale (nessuna modifica)

2.6.2 Variabili d'ambiente

Tabella 2.11: Variabili d'ambiente introdotte dalla componente RAG.

Variabile	Valore di default	Descrizione
QDRANT_URL	http://localhost:6333	Endpoint REST Qdrant
QDRANT_COLLECTION_NAME	i3guild_knowledge	Collection principale
OLLAMA_BASE_URL	http://localhost:11434	Endpoint Ollama per generazione LLM
OLLAMA_LLM_MODEL / OLLAMA_CHAT_MODEL	gemma3:1b / configurabile	Modello LLM usato dal servizio o inoltrato dal frontend
EMBEDDING_DIMENSIONS	768	Dimensione vettore denso
FASTEMBED_DENSE_MODEL	BAAI/bge-base-en-v1.5	Modello FastEmbed denso
FASTEMBED_SPARSE_MODEL	prithivida/Splade_PP_en.v1	Modello FastEmbed sparso
RAG_CACHE_TTL_SECONDS	60	TTL cache retrieval in-process
RAG_API_URL	http://i3guild_rag:8000	Endpoint interno del servizio RAG

2.6.3 Integrazione lato Next.js

Lato applicazione Next.js, l'integrazione è concentrata nel servizio TypeScript `src/lib/services/rag.service.ts`. Il client espone funzioni per retrieval, risposta sincrona e streaming, inoltra al microservizio il modello LLM configurato e supporta opzioni applicative come `use_rag`, `json_mode`, `timeout_client` e parametri di generazione. In caso di errore del retrieval, la ricerca può degradare restituendo un insieme vuoto; nei flussi di generazione strutturata, invece, l'errore viene propagato al chiamante per attivare fallback applicativi espliciti.

2.6.4 Sincronizzazione dei dati

PostgreSQL rimane la sorgente autoritativa; Qdrant è una vista derivata e deve essere mantenuto sincronizzato con i dati applicativi. Sono state considerate tre strategie, in ordine crescente di complessità: re-indicizzazione completa, sincronizzazione incrementale e aggiornamento event-driven. Per il prototipo la strategia raccomandata è il **full re-index periodico**: il container **i3guild-embedding-pipeline** ricrea la collection di default e riscrive i documenti con policy di overwrite. La scelta è semplice da implementare, riduce il rischio di inconsistenze tra indici densi e sparsi e rimane sostenibile perché l'embedding è eseguito localmente, senza costi a consumo per token. L'evoluzione verso un *sync incrementale* basato sul campo `@updatedAt` di Prisma, o verso un meccanismo *event-driven* attivato dalle operazioni CRUD, è lasciata come sviluppo futuro.

Capitolo 3

Implementazione del Sistema RAG

Il capitolo descrive l'implementazione del sistema RAG integrato in i3Guild. L'obiettivo non è ripetere le scelte architetturali già motivate nel Capitolo 2, ma mostrare come tali scelte siano state tradotte in componenti eseguibili: una pipeline di ingestione per popolare Qdrant, un microservizio FastAPI per il retrieval e la generazione, un'integrazione applicativa lato Next.js e una generazione controllata delle bozze di nuove Gilde. Durante lo sviluppo è stato anche implementato ed esplorato un workflow agentico; viene documentato come soluzione sperimentale, ma non come scelta finale, perché a parità di risorse hardware introduceva un costo computazionale più elevato e un vantaggio limitato rispetto a un flusso RAG più classico.

La realizzazione è distribuita su tre repository applicativi. Il primo è **i3guild**, che contiene l'applicazione web Next.js e il database PostgreSQL [53]. Il secondo è **i3guild-embedding-pipeline**, responsabile dell'estrazione dei dati, della normalizzazione testuale, della generazione di embedding densi e sparsi con FastEmbed/Haystack e dell'upsert su Qdrant. Il terzo è **i3guild-rag**, un microservizio Python che espone le API di retrieval e chat. Questa separazione riduce l'accoppiamento tra applicazione e componente RAG: i3Guild rimane la sorgente autoritativa dei dati, Qdrant conserva la knowledge base ibrida e Ollama viene usato solo per la generazione LLM [55, 48].

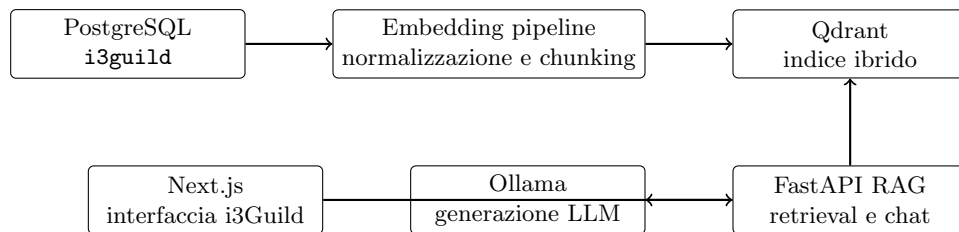


Figura 3.1: Flusso implementativo end-to-end tra sorgente dati, indicizzazione ibrida, retrieval e generazione.

3.1 Struttura dei repository e responsabilità

La suddivisione dei moduli segue il flusso dei dati. PostgreSQL contiene le Gilde, le epoche e le relazioni con gli utenti; la pipeline di embedding legge queste informazioni e produce documenti Haystack [40]; Qdrant memorizza vettori densi, rappresentazioni sparse e metadati; il microservizio RAG interroga l'indice ibrido e, quando richiesto, invoca Ollama per generare la risposta. L'applicazione Next.js consuma il servizio tramite un client HTTP dedicato [60].

Tabella 3.1: Componenti implementativi principali.

Repository / modulo	File principali	Responsabilità
i3guild	services/rag.service.ts	Client TypeScript verso il microservizio RAG; gestione dello streaming e degradazione controllata in caso di errore.
i3guild-embedding-pipeline	pipeline/main.py	Estrazione da PostgreSQL, normalizzazione, chunking, embedding denso e sparso con FastEmbed e scrittura su Qdrant.
i3guild-rag	app/main.py, app/rag.py, app/pipeline.py, app/ollama.service.py	API FastAPI, pipeline Haystack di query ibrida, retrieval dinamico, chiamate a Ollama e streaming NDJSON.
i3guild-rag/app/agent	orchestrator.py, state.py, models.py	Workflow agentic deterministico esplorato per ricerca, ideazione e compilazione progressiva della proposta di Gilda; non adottato nella soluzione finale.

La scelta di mantenere l'ingestione fuori dal servizio di query evita che operazioni batch e richieste interattive competano nello stesso processo. La pipeline di embedding può essere eseguita manualmente, schedata o avviata in container dedicato; il servizio RAG, invece, rimane ottimizzato per rispondere a richieste HTTP a bassa latenza. La containerizzazione isola dipendenze e servizi di supporto [41].

3.2 Pipeline di ingestione

La pipeline di ingestione è implementata in `i3guild-embedding-pipeline/pipeline/main.py`. Il processo viene avviato come job: verifica le dipendenze, legge le sorgenti dati, produce documenti Haystack, genera embedding densi e sparsi tramite FastEmbed e scrive i risultati in Qdrant [40, 55]. In caso di errore in una dipendenza essenziale, ad esempio PostgreSQL o Qdrant non raggiungibile, il processo termina con codice diverso da zero. Questo comportamento è utile in ambiente Docker o CI, perché rende immediatamente visibile il fallimento dell'indicizzazione [41].

3.2.1 Configurazione e controlli preliminari

La configurazione avviene tramite variabili d'ambiente. Le più rilevanti sono `DB_URL`, `QDRANT_URL`, `QDRANT_COLLECTION_NAME`, `FASTEMBED_DENSE_MODEL`, `FASTEMBED_SPARSE_MODEL` ed `EMBEDDING_DIMENSIONS`. Il modulo legge anche parametri operativi come `RECREATE_INDEX`, `INGEST_PIPELINE_BATCH_SIZE`, `INGEST_MAX_DOCUMENT_CHARS` e `INGEST_CHUNK_OVERLAP_CHARS`. La pipeline non richiede più `OLLAMA_BASE_URL` o `OLLAMA_EMBEDDING_MODEL` per l'indicizzazione.

Tabella 3.2: Parametri principali della pipeline di ingestione.

Variabile	Ambito	Uso nell'implementazione
<code>DB_URL</code>	PostgreSQL	Connessione alla sorgente autoritativa delle Gilde.
<code>QDRANT_URL</code>	Qdrant	Endpoint del vector database usato in scrittura.
<code>QDRANT_COLLECTION_NAME</code>	Qdrant	Nome della collection da creare o aggiornare.
<code>FASTEMBED_DENSE_MODEL</code>	FastEmbed	Modello usato per gli embedding densi.
<code>FASTEMBED_SPARSE_MODEL</code>	FastEmbed	Modello usato per gli embedding sparsi.
<code>FASTEMBED_CACHE_DIR</code>	FastEmbed	Directory locale per la cache dei modelli.
<code>EMBEDDING_DIMENSIONS</code>	Indice vettoriale	Dimensione dei vettori densi salvati in Qdrant.
<code>RECREATE_INDEX</code>	Operatività	Ricrea la collection; nella versione aggiornata il default è <code>true</code> .
<code>INGEST_PIPELINE_BATCH_SIZE</code>	Operatività	Numero di documenti elaborati per run Haystack.
<code>INGEST_MAX_DOCUMENT_CHARS</code>	Chunking	Limite oltre il quale un documento viene suddiviso.

Prima dell'ingestione vengono eseguiti controlli sulle configurazioni essenziali: connessione a PostgreSQL, configurazione FastEmbed, dimensione del vettore denso e raggiungibilità della collection Qdrant tramite il document store Haystack. La collection viene creata con distanza coseno, dimensione pari a `EMBEDDING_DIMENSIONS` e `use_sparse_embeddings=true`. Il valore usato per la componente densa è 768.

3.2.2 Estrazione dei dati da PostgreSQL

La funzione `fetch_all_guilds()` estrae le informazioni semantiche delle Gilde tramite una query SQL con join su `Epoch`, `GuildUserRelation` ed `ExternalUser`. Il risultato include i campi testuali principali della Gilda, i riferimenti all'epoca, la modalità di partecipazione, l'indicazione di Gilda individuale e la lista dei partecipanti. Il listato completo dei campi estratti è riportato in Appendice B, Listato B.4.

La query reale include anche una sottoquery per aggregare i partecipanti, combinando utenti interni ed esterni. Questo passaggio è importante perché i nomi dei partecipanti sono utili sia come metadati filtrabili sia come contesto testuale per le domande dell'utente.

3.2.3 Normalizzazione e costruzione dei documenti

Ogni riga estratta viene trasformata in uno o più oggetti `haystack.Document`. La funzione `create_documents_from_row()` compone un documento testuale con sezioni marcate, ad esempio `[SOMMARIO]`, `[OBIETTIVI]`, `[RISCHI]`, `[RISULTATI RAGGIUNTI]` e `[PARTECIPANTI]`. I campi HTML vengono normalizzati con `BeautifulSoup` [58]: il codice rimuove i tag, conserva il testo e compatta gli spazi. Per il retrieval serve un testo pulito e stabile, non una resa editoriale. Uno schema semplificato della costruzione del documento è riportato in Appendice B, Listato B.5.

La strategia descritta nel Capitolo 2 prevedeva un documento consolidato per Gilda. L'implementazione mantiene questa impostazione come caso normale, ma introduce una soglia di sicurezza: `INGEST_MAX_DOCUMENT_CHARS`. Se il testo supera il limite, la funzione `split_text_with_overlap()` lo divide in chunk con sovrapposizione controllata. In questo modo gli outlier non impediscono l'ingestione e, allo stesso tempo, i chunk successivi mantengono il riferimento alla Gilda tramite l'intestazione. La granularità dei chunk condiziona qualità del retrieval e quantità di contesto disponibile per il modello generativo [8].

Gli identificativi dei documenti sono deterministici: `guild-{guild_id}-chunk-{index}`. La scelta rende idempotente la reindicizzazione: rieseguire la pipeline sulla stessa Gilda produce gli stessi ID e permette a Qdrant di sovrascrivere i punti esistenti invece di duplicarli. I metadati associati includono `guild_id`, `guild_name`, `epoch_id`, `is_individual`, `participation_mode`, `participants`, `chunk_index` e `chunk_count`.

3.2.4 Embedding e scrittura in Qdrant

La fase finale costruisce una pipeline Haystack composta da `FastembedDocumentEmbedder`, `FastembedSparseDocumentEmbedder` e `DocumentWriter`. Il primo componente aggiunge ai documenti la rappresentazione densa; il secondo aggiunge la rappresentazione sparsa; il writer persiste entrambi i segnali nel `QdrantDocumentStore`. La configurazione essenziale della pipeline è riportata in Appendice B, Listato B.6.

La pipeline viene eseguita in batch applicativi, configurati tramite `INGEST_PIPELINE_BATCH_SIZE`. Questa scelta riduce il picco di memoria dello sparse embedder e mantiene il comportamento batch del container: il job parte, scrive su Qdrant e termina. La normalizzazione dei campi avviene prima della costruzione dei documenti, rendendo esplicito quali testi entrano nella knowledge base.

3.3 Microservizio RAG

Il microservizio RAG è implementato con FastAPI nel repository `i3guild-rag`. Espone endpoint per retrieval, chat sincrona e chat in streaming. La configurazione è centralizzata in `app/settings.py` tramite `pydantic-settings`; i valori di default permettono lo sviluppo locale, mentre gli ambienti Docker possono sovrascriverli con variabili d'ambiente [42, 54].

3.3.1 Avvio, middleware e gestione errori

Il file `app/main.py` definisce l'applicazione FastAPI e registra CORS, rate limiting e handler globale delle eccezioni. Il rate limiter usa `slowapi` con limite predefinito di 100 richieste al minuto per IP sugli endpoint principali. Il sistema non tratta tutti gli errori allo stesso modo: se il messaggio suggerisce una dipendenza non raggiungibile, ad esempio Qdrant o Ollama, la risposta HTTP è 503; negli altri casi viene restituito 500.

Tabella 3.3: Endpoint principali esposti dal microservizio RAG.

Endpoint	Formato	Responsabilità
POST <code>/rag/retrieve</code>	JSON	Recupera documenti semanticamente pertinenti e restituisce metadati di diagnostica sul taglio dei risultati.
POST <code>/chat/reply</code>	JSON	Esegue retrieval, costruisce il prompt aumentato e restituisce una risposta completa.
POST <code>/chat/stream</code>	NDJSON	Espone la chat incrementale, separando eventi RAG, token, warning ed errori.

Per ridurre il rischio operativo in ambienti con risorse limitate, il servizio controlla la memoria disponibile prima delle chiamate a modelli locali. La funzione `get_model_memory_requirement()` stima il requisito RAM in base al nome del modello; `get_ram_warning()` produce un warning se la memoria libera è inferiore alla soglia. Il warning non blocca la richiesta, ma viene propagato nella risposta quando disponibile.

3.3.2 Endpoint di retrieval

L'endpoint POST `/rag/retrieve` riceve una query, parametri di retrieval e filtri opzionali. Il modello Pydantic `RetrieveRequest` definisce i valori di default: `top_k=20`, `score_threshold=0.7`, `max_documents=10` e `gap_threshold=0.08`. La risposta include i documenti selezionati, il numero di candidati iniziali, la soglia usata, il motivo del taglio, l'eventuale query riscritta e gli eventuali filtri estratti automaticamente. Lo schema della richiesta è riportato in Appendice B, Listato B.7.

L'endpoint è volutamente sottile: valida l'input, esegue il controllo RAM e delega la logica a `retrieve_documents()`. Questa separazione rende più semplice testare il retrieval senza dover passare da FastAPI.

3.3.3 Pipeline Haystack di query

La pipeline di query è costruita in `app/pipeline.py`. Il `QdrantDocumentStore` punta alla collection configurata, usa embedding densi di dimensione 768 con similarità coseno e abilita `use_sparse_embeddings`. La query utente viene elaborata da due componenti: `FastembedTextEmbedder`, che produce l'embedding denso, e `FastembedSparseTextEmbedder`, che produce la rappresentazione sparsa. I due output alimentano `QdrantHybridRetriever`, responsabile della ricerca ibrida in Qdrant [40, 55]. Il listato della pipeline è riportato in Appendice B, Listato B.8.

La pipeline è inizializzata in modo lazy e protetta da un lock. Alla prima richiesta viene creato l'oggetto Haystack e vengono predisposti gli indici sui payload Qdrant per i campi usati più spesso nei filtri: `epoch_id`, `is_individual`, `participation_mode`, `participants` e `guild_name`. Le chiamate successive riusano la stessa pipeline in memoria. La ricerca vettoriale su Qdrant usa indici approssimati per nearest neighbor; HNSW è il riferimento adottato per questa classe di problemi [23].

3.4 Logica di retrieval

La funzione `retrieve_documents()` in `app/rag.py` contiene la parte più specifica del sistema. Il retrieval non è una semplice chiamata a Qdrant: prima della ricerca la query viene normalizzata e, quando possibile, riscritta; dopo la ricerca i risultati vengono deduplicati e filtrati con una logica dinamica. Questa struttura segue il principio RAG di separare conoscenza parametrica e recupero esplicito di documenti esterni [21].

3.4.1 Riscrittura della query e filtri

La funzione `extract_topic_query()` rimuove parole frequenti nel dominio i3Guild, come "gilda", "fondare" o "consigli", quando non contribuiscono al tema ricercato. Se dalla frase "vorrei fondare una gilda sulla musica" viene estratto il topic "musica", la ricerca viene eseguita su tale topic. La risposta conserva la query riscritta nel campo `topic_query`, così il frontend può mostrarla o usarla per diagnostica.

In parallelo, `extract_filters_from_query()` cerca vincoli espressi in linguaggio naturale: epoca, Gilde individuali o di gruppo, modalità di partecipazione e partecipanti. Se l'utente passa anche filtri espliciti, il servizio li combina con quelli estratti tramite un operatore **AND**. Questo permette di usare la ricerca vettoriale senza perdere vincoli strutturati presenti nei metadati.

3.4.2 Cache e deduplicazione

Per evitare interrogazioni ripetute identiche, `retrieve_documents()` mantiene una cache in memoria con TTL configurabile tramite `rag_cache_ttl_seconds`. La chiave include query effettiva, parametri di retrieval e filtri. Si tratta di una cache locale al processo, sufficiente per lo scenario prototipale; in un deployment multi replica andrebbe sostituita con una cache condivisa o disabilitata per evitare differenze tra istanze.

Dopo il recupero da Qdrant, il servizio rimuove duplicati su tre livelli. Prima controlla l'ID del documento, poi usa una fingerprint testuale normalizzata e infine confronta i testi con `difflib.SequenceMatcher`. La soglia fuzzy è configurabile con `rag_dedup_fuzzy_threshold`. Questa parte è stata aggiunta perché il corpus può contenere documenti equivalenti provenienti da sorgenti diverse o chunk molto simili generati dalla sovrapposizione.

3.4.3 Selezione dinamica dei documenti

La selezione finale avviene in `filter_documents_dynamically()`. I documenti vengono ordinati per score, filtrati con una soglia minima e poi tagliati quando il calo relativo tra due score consecutivi supera `gap_threshold`. Se il numero di risultati resta troppo alto, viene applicato `max_documents`. La risposta indica il motivo del taglio: `threshold`, `relative_gap`, `max_cap` o `exhausted`. Uno schema della logica di taglio è riportato in Appendice B, Listato B.9.

Questa scelta evita un limite `top_k` fisso. Per query molto specifiche il sistema restituisce pochi documenti; per query più ampie può mantenere più contesto, entro il cap configurato. Il risultato è più stabile per l'utente e riduce la probabilità di passare al modello generativo documenti debolmente pertinenti. Ridurre contesto non pertinente è utile anche per mitigare risposte non fondate sulle fonti recuperate [26].

3.5 Generazione con Ollama

Il modulo `app/ollama_service.py` gestisce le chiamate al modello di chat. Le funzioni principali sono `generate_chat_reply()` per risposte complete e `generate_chat_reply_stream()` per risposte in streaming. Entrambe usano l'endpoint `/api/chat` di Ollama [48].

3.5.1 Risoluzione del modello

Il modello configurato in `OLLAMA_LLM_MODEL` viene confrontato con la lista dei modelli installati ottenuta da `/api/tags`. Il codice accetta sia corrispondenze esatte sia varianti con tag, ad esempio `:latest`. Se la configurazione è ambigua o il modello non è installato, il servizio restituisce un errore esplicito. Questa verifica evita fallimenti meno leggibili durante la generazione.

La lista dei modelli disponibili viene mantenuta in cache per pochi secondi (`ollama_model_cache_ttl_seconds`), così il servizio non interroga Ollama a ogni richiesta. La cache è locale al processo e ha solo funzione di riduzione dell'overhead.

3.5.2 Costruzione del prompt aumentato

Prima di inviare la richiesta a Ollama, `_prepare_messages()` costruisce la lista dei messaggi. Il system prompt, se presente, viene aggiunto per primo. La cronologia conversazionale viene limitata agli ultimi `max_history_messages`, in modo da controllare la dimensione del contesto. Se il retrieval ha restituito documenti, il testo viene aggiunto al messaggio utente dentro una sezione esplicita: `--- CONTESTO RAG ---`. L'aumento del prompt con documenti recuperati è il meccanismo con cui RAG rende disponibili al modello informazioni esterne senza modificare i pesi [21]. Il formato del messaggio aumentato è riportato in Appendice B, Listato B.10.

Ogni documento viene troncato a `rag_max_chars_per_doc` caratteri. Il troncamento è intenzionale: una singola Gilda molto lunga non deve consumare tutta la finestra di contesto né impedire al modello di vedere altre fonti.

3.5.3 Streaming NDJSON

Per la chat interattiva il servizio espone `POST /chat/stream`. Prima invia un evento `rag` con i metadati del retrieval, poi inoltra gli eventi generati da Ollama. La risposta HTTP usa il media type `application/x-ndjson`; ogni riga è un oggetto JSON indipendente. Questo formato è semplice da consumare lato frontend e permette di distinguere token, warning, errori e fine stream. Un esempio di sequenza NDJSON è riportato in Appendice B, Listato B.11.

3.6 Workflow agentic

Durante il progetto è stata esplorata anche una modalità agentic per guidare l'utente nella proposta di nuove Gilde. Questa linea non costituisce però il flusso finale del prototipo: la versione aggiornata mantiene il servizio RAG centrato su retrieval ibrido, chat e generazione controllata, mentre la compilazione della bozza viene gestita lato applicazione attraverso prompt strutturati e validazione del risultato.

La scelta è coerente con i vincoli osservati durante lo sviluppo. Un agente che mantiene stato, pianifica passaggi e invoca più volte il modello introduce maggiore latenza e maggiore consumo

di memoria, soprattutto con LLM locali di piccole dimensioni. Il flusso finale privilegia quindi un approccio più deterministico: il retrieval recupera contesto quando serve; la generazione strutturata viene attivata solo per produrre una bozza; la validazione resta nel codice applicativo. La componente agentica rimane una possibile direzione futura, da rivalutare con modelli locali più capaci o con hardware più ampio.

3.7 Integrazione lato Next.js

L'applicazione `i3Guild` accede al microservizio tramite `src/lib/services/rag.service.ts`. Il client centralizza l'URL del servizio, legge `I3GUILD_RAG_URL` o `RAG_API_URL`, serializza le richieste e normalizza gli errori. Questa scelta evita di distribuire chiamate HTTP dirette al servizio RAG in più componenti React o route API [60].

3.7.1 Client di retrieval

La funzione `searchRAGWithMeta()` invoca `/rag/retrieve` e restituisce sia documenti sia metadati. In caso di errore non solleva un'eccezione: restituisce una lista vuota e un `cutoff_reason` pari a `error`. Per la sola ricerca semantica questo comportamento è corretto: il chatbot non deve bloccarsi se il servizio Python è temporaneamente non raggiungibile. La chiamata TypeScript essenziale è riportata in Appendice B, Listato B.14.

La funzione `searchRAG()` è un wrapper più semplice che restituisce solo i documenti. Viene mantenuta per i punti dell'applicazione che non hanno bisogno della diagnostica.

3.7.2 Client chat e generazione controllata

Per la generazione sono disponibili `generateRAGChatReply()`, `generateRAGChatStream()` e le funzioni di retrieval. La prima usa l'endpoint non streaming `/chat/reply`; la seconda restituisce direttamente la `Response`, lasciando al chiamante la lettura incrementale dello stream NDJSON. Il client può inoltrare il modello LLM configurato, attivare o disattivare il retrieval con `use_rag`, richiedere JSON mode e passare opzioni di generazione.

Questa interfaccia consente di riusare lo stesso microservizio in due modi distinti: chat RAG, con retrieval attivo, e generazione strutturata, in cui il prompt contiene già tutto il contesto necessario e il retrieval viene saltato. La distinzione riduce accoppiamento tra caso d'uso conversazionale e generazione di bozze.

3.8 Generazione della bozza di Gilda

Accanto alla chat RAG, `i3Guild` include un flusso per generare una bozza di Gilda a partire dalla conversazione. La funzionalità è documentata come *Guild Draft Generation* e si attiva dal pulsante "Proponi bozza" nella chat. Il flusso non crea direttamente una nuova Gilda e non invia l'utente alla pagina di creazione: produce invece un oggetto strutturato che rappresenta un draft, modificabile e verificabile prima di qualunque salvataggio applicativo.

Il formato atteso è `GuildDraft`. Include nome, sommario, obiettivi, risultati attesi, rischi, opportunità, budget, risorse, journal, modalità di partecipazione, numero minimo e massimo di partecipanti, indicazione di Gilda individuale e avatar. La route Next.js `/api/guild/draft` recupera

la conversazione, costruisce un prompt istruttivo, chiama il microservizio RAG con `use_rag=false` e `json_mode=true`, quindi prova a estrarre un JSON compatibile con lo schema. La validazione è quindi parte del contratto applicativo: una risposta testuale plausibile, ma non riconducibile allo schema atteso, non viene trattata come bozza valida.

La gestione degli errori è intenzionalmente esplicita. Se la conversazione non è sufficiente, se il modello restituisce JSON non valido o se mancano campi obbligatori, il sistema informa l'utente del problema e lo invita a riprovare, eventualmente aggiungendo dettagli più chiari nella conversazione. Questo comportamento rende misurabile il tasso di fallimento della generazione: in una valutazione applicativa, tale tasso può essere espresso come rapporto tra bozze non valide e numero totale di richieste di generazione.

La risposta della route è NDJSON streaming: invia eventi di stato periodici mentre il modello lavora e conclude con un evento `draft` oppure con un evento `error`. Questa scelta evita che un gateway con timeout aggressivo interrompa operazioni LLM lente prima che il frontend riceva un esito interpretabile.

3.9 Aspetti operativi

L'implementazione include alcune scelte operative che non cambiano la logica RAG ma ne migliorano l'affidabilità durante l'uso locale.

3.9.1 Timeout e dipendenze lente

Ollama può impiegare diversi secondi, o più di un minuto su CPU, per caricare un modello non ancora residente in memoria. Il problema si presenta soprattutto alla prima richiesta dopo l'avvio del container: il modello è installato, ma il processo deve ancora caricarlo e inizializzare l'inferenza. Per questo il servizio RAG usa `OLLAMA_REQUEST_TIMEOUT_SECONDS` per le chiamate sincrone e consente un override per richiesta tramite `timeout_seconds`. Nei flussi lunghi, un valore minore o uguale a zero disabilita il timeout HTTP verso Ollama.

La stessa attenzione serve lato Next.js. La generazione della bozza usa una risposta NDJSON streaming con heartbeat periodici, così la route rimane viva mentre il modello produce il JSON. Il frontend distingue eventi di stato, bozza valida ed errori, invece di trattare ogni ritardo come fallimento indistinto.

3.9.2 System prompt e modello derivato

Una seconda ottimizzazione operativa riguarda la gestione del system prompt. Nella configurazione standard il prompt di sistema viene inviato a ogni richiesta insieme alla conversazione e al contesto recuperato. Per ridurre la duplicazione del payload, il progetto considera anche la creazione di un modello Ollama derivato tramite l'endpoint `/api/create`, incorporando il system prompt nella definizione del modello. Il nome del modello derivato include un hash del prompt, così una modifica del prompt produce una nuova configurazione senza sovrascrivere quella precedente.

Questa soluzione migliora soprattutto la stabilità configurativa e riduce la quantità di testo serializzata nelle richieste. Non va però interpretata come un riutilizzo effettivo della KV Cache del modello: il system prompt incorporato nella definizione del modello rende più pulita la configurazione, ma non elimina di per sé il costo computazionale del prompt durante l'inferenza.

3.9.3 Tracciabilità

Il servizio espone dati diagnostici in più punti: `cutoff_reason`, `topic_query`, `extracted_filters`, warning RAM, numero di candidati, documenti restituiti e log di timeout verso Ollama. Questi campi non sono solo utili per il debug. Permettono di ricostruire il comportamento del sistema durante la valutazione applicativa: quale query è stata effettivamente cercata, quanti documenti sono stati recuperati, perché alcuni risultati sono stati scartati e se il collo di bottiglia si trova nel retrieval o nella generazione.

3.10 Sintesi del capitolo

Il sistema implementato combina componenti semplici, ma separati con precisione. La pipeline di embedding prepara l'indice ibrido a partire da PostgreSQL; il microservizio RAG espone retrieval e generazione; Qdrant gestisce vettori densi, rappresentazioni sparse e metadati; Ollama mantiene l'inferenza generativa in locale; Next.js integra chat, generazione bozza e fallback applicativi. Le parti più specifiche del lavoro sono la normalizzazione del corpus i3Guild, il retrieval ibrido con query rewriting, cache e deduplicazione, e la separazione tra chat RAG e generazione strutturata senza retrieval. Il workflow agentico è stato trattato come una linea sperimentale e non come soluzione finale, perché a parità di hardware introduceva maggiore complessità e maggiore costo computazionale. Queste scelte rendono il prototipo coerente con i vincoli di privacy e di risorse discussi nei capitoli precedenti, lasciando aperti margini di evoluzione su sincronizzazione incrementale, agenti più flessibili e valutazione sistematica della qualità delle risposte.

L'esperienza aziendale porta anche a un problema più generale, che supera il solo caso i3Guild: quando un sistema RAG deve essere eseguito localmente, la qualità applicativa dipende dalla sostenibilità dell'inferenza. Non basta che il modello sia disponibile in locale; occorre capire se entra nella memoria disponibile, quanto costa caricarlo, quanto tempo impiega a produrre il primo token e quale throughput mantiene con prompt di lunghezza diversa. Da questa esigenza nasce la seconda parte della tesi. I Capitoli ?? e ?? isolano questo tema dal caso aziendale e lo studiano in modo sistematico su Apple Silicon, usando MLX, più modelli e più livelli di quantizzazione.

Conclusioni e Sviluppi Futuri

Le conclusioni sintetizzano il percorso svolto: dallo studio delle tecniche RAG e dei modelli locali, alla progettazione del sistema per i3Guild, fino all'implementazione del prototipo e alla valutazione delle sue configurazioni principali. Il contributo centrale della tesi non consiste nell'addestramento di un nuovo modello, ma nell'integrazione di un sistema RAG enterprise on-premise in un contesto aziendale reale, con vincoli di privacy, risorse e manutenibilità.

La sezione conclusiva dovrà evidenziare tre risultati. Primo, la possibilità di usare il corpus delle Gilde come base di conoscenza interrogabile in linguaggio naturale. Secondo, il ruolo delle ottimizzazioni non parametriche, come query rewriting, filtri strutturati, deduplicazione e soglie dinamiche, nel rendere il retrieval più adatto al dominio. Terzo, il valore e i limiti dell'estensione agentica, utile per guidare l'utente nella proposta di nuove Gilde ma ancora dipendente dalla qualità del corpus e dal comportamento dei modelli locali.

I limiti principali riguardano l'assenza di un glossario aziendale validato, la mancanza di una ground truth estesa, la dimensione ridotta dei modelli eseguibili localmente e la natura preliminare delle sperimentazioni su tecniche di adattamento più invasive. Questi limiti non invalidano il prototipo, ma definiscono con precisione il perimetro entro cui interpretarne i risultati.

Gli sviluppi futuri possono essere organizzati su tre direzioni: arricchimento incrementale della base di conoscenza, costruzione di un dataset di valutazione più ampio e validato con stakeholder aziendali, e sperimentazione più approfondita di strategie come fine-tuning leggero, LoRA/QLoRA o orchestrazione agentica più flessibile. Tali estensioni potranno essere considerate solo dopo aver consolidato il sistema RAG di base e il processo di valutazione.

Appendice A

Annotazioni e Convenzioni

Questa appendice raccoglie le convenzioni usate nella descrizione del sistema RAG. L'obiettivo è evitare ripetizioni nei capitoli principali e rendere esplicito il significato di nomi, formati e parametri ricorrenti.

A.1 Repository e componenti

Tabella A.1: Convenzioni di denominazione dei repository.

Nome	Significato
i3guild	Applicazione web principale, basata su Next.js, PostgreSQL e Keycloak.
i3guild-embedding-pipeline	Job Python che estrae, normalizza e indicizza il corpus delle Gilde.
i3guild-rag	Microservizio FastAPI che espone retrieval ibrido, chat RAG e generazione controllata.

A.2 Formato dei documenti indicizzati

I documenti salvati in Qdrant derivano dai record applicativi di i3Guild. Ogni documento contiene un testo normalizzato e un insieme di metadati usati per filtri, diagnostica e deduplicazione.

A.3 Eventi NDJSON

Gli endpoint streaming restituiscono una sequenza di righe JSON indipendenti. La scelta del formato NDJSON permette al frontend di elaborare subito i metadati RAG e poi aggiornare l'interfaccia man mano che arrivano i token.

Tabella A.2: Metadati principali associati ai documenti indicizzati.

Campo	Uso
<code>guild_id</code>	Identificativo della Gilda sorgente.
<code>guild_name</code>	Nome della Gilda, usato anche nei risultati mostrati all'utente.
<code>epoch_id</code>	Identificativo dell'epoca a cui appartiene la Gilda.
<code>participants</code>	Lista dei partecipanti, utile come metadato filtrabile.
<code>participation_mode</code>	Modalità di partecipazione, ad esempio in presenza, remota o ibrida.
<code>is_individual</code>	Indica se la proposta riguarda una Gilda individuale.
<code>chunk_index</code>	Posizione del chunk quando una Gilda viene suddivisa.
<code>chunk_count</code>	Numero totale di chunk generati per la stessa Gilda.

Tabella A.3: Tipi di evento usati dagli endpoint streaming.

Evento	Significato
<code>rag</code>	Contiene documenti recuperati, filtri, query riscritta e motivo del taglio dei risultati.
<code>status</code>	Indica l'avanzamento di una generazione asincrona lato applicazione.
<code>stream_start</code>	Segnala l'inizio della risposta generata.
<code>token</code>	Contiene un frammento incrementale della risposta.
<code>stream_end</code>	Chiude lo stream dopo l'ultimo token generato.
<code>error</code>	Comunica un errore gestibile senza interrompere il contratto dello stream.

Appendice B

Codice Utilizzato

Questa appendice raccoglie i listati richiamati nei capitoli principali. Nel testo sono mantenute le scelte progettuali, le decisioni implementative e il razionale; i frammenti di codice sono spostati qui per non interrompere la lettura.

B.1 Listati richiamati nel Capitolo 2

Listing B.1: Struttura della pipeline di ingestione Haystack 2.x (pseudocodice).

```
from haystack import Pipeline
from haystack.components.writers import DocumentWriter
from haystack.integrations.components.embedders.fastembed import (
    FastembedDocumentEmbedder,
    FastembedSparseDocumentEmbedder,
)
from haystack.integrations.document_stores.qdrant import QdrantDocumentStore

qdrant_store = QdrantDocumentStore(url="http://qdrant:6333",
                                   index="i3guild_knowledge",
                                   embedding_dim=768,
                                   use_sparse_embeddings=True)

indexing_pipeline = Pipeline()
indexing_pipeline.add_component("dense_embedder",
                               FastembedDocumentEmbedder(
                                   model="BAAI/bge-base-en-v1.5"))
indexing_pipeline.add_component("sparse_embedder",
                               FastembedSparseDocumentEmbedder(
                                   model="prithivida/Splade_PP-en-v1"))
indexing_pipeline.add_component("writer", DocumentWriter(
    document_store=qdrant_store,
```

```

        policy=DuplicatePolicy.OVERWRITE))

indexing_pipeline.connect("dense_embedder.documents", "sparse_embedder.
    documents")
indexing_pipeline.connect("sparse_embedder.documents", "writer.
    documents")

indexing_pipeline.run({"dense_embedder": {"documents": raw_docs}})

```

Listing B.2: Esempio di request body per l'endpoint `/rag/retrieve`.

```

{
  "query":          "vorrei fondare una gilda sulla musica",
  "top_k":          20,
  "score_threshold": 0.5,
  "max_documents":  10,
  "gap_threshold":  0.08,
  "filters":        null
}

```

Listing B.3: Struttura della response dell'endpoint `/rag/retrieve`.

```

{
  "documents":      [ { "content": "...", "score": 0.85,
                        "meta": { "guild_name": "...",
                                  "participants": [...] } } ],
  "total_candidates": 15,
  "threshold_used":  0.5,
  "cutoff_reason":   "relative_gap",
  "topic_query":     "musica"
}

```

B.2 Listati richiamati nel Capitolo 3: pipeline di ingestione

Listing B.4: Campi principali estratti da PostgreSQL per l'ingestione.

```

SELECT
  g.id AS guild_id ,
  g.name AS guild_name ,
  g.summary ,
  g.goals ,
  g.opportunities ,
  g.outcome ,
  g.risks ,
  g."otherResources" AS other_resources ,

```

```

    g.budget ,
    g."achievedResultsDescription" AS achieved_results_description ,
    g.journal ,
    g."isIndividualGuild" AS is_individual ,
    g."participationMode" AS participation_mode ,
    e.id AS epoch_id ,
    e."startDate" AS epoch_start ,
    e."endDate" AS epoch_end
FROM "Guild" g
LEFT JOIN "Epoch" e ON g."epochId" = e.id

```

Listing B.5: Schema semplificato della costruzione del documento testuale.

```

parts = [f"[GUILD: {guild_name}]"]

if row.get("summary"):
    parts.append(f"[SOMMARIO] {clean_html(row.get('summary'))}")
if row.get("goals"):
    parts.append(f"[OBIETTIVI] {clean_html(row.get('goals'))}")
if row.get("risks"):
    parts.append(f"[RISCHI] {clean_html(row.get('risks'))}")
if participants:
    parts.append(f"[PARTECIPANTI] {' , - '.join(participants)}")

full_text = "\n".join(parts)

```

Listing B.6: Pipeline Haystack usata per embedding e scrittura.

```

document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=EMBEDDING_DIMENSIONS,
    similarity="cosine",
    recreate_index=RECREATE_INDEX,
    use_sparse_embeddings=True,
)

dense_embedder = FastembedDocumentEmbedder(
    model=FASTEMBED_DENSE_MODEL,
    cache_dir=FASTEMBED_CACHE_DIR,
)
sparse_embedder = FastembedSparseDocumentEmbedder(
    model=FASTEMBED_SPARSE_MODEL,
    cache_dir=FASTEMBED_CACHE_DIR,
)

writer = DocumentWriter(document_store=document_store, policy=policy)

```



```
indexing_pipeline = Pipeline()
indexing_pipeline.add_component("dense_embedder", dense_embedder)
indexing_pipeline.add_component("sparse_embedder", sparse_embedder)
indexing_pipeline.add_component("writer", writer)
indexing_pipeline.connect("dense_embedder.documents", "sparse_embedder.
    documents")
indexing_pipeline.connect("sparse_embedder.documents", "writer.
    documents")
```

B.3 Listati richiamati nel Capitolo 3: retrieval e generazione

Listing B.7: Schema Pydantic della richiesta di retrieval.

```
class RetrieveRequest(BaseModel):
    query: str
    top_k: int = 20
    score_threshold: float = 0.7
    max_documents: int = 10
    gap_threshold: float = 0.08
    filters: dict | None = None
```

Listing B.8: Pipeline Haystack per la ricerca ibrida.

```
document_store = QdrantDocumentStore(
    url=QDRANT_URL,
    index=QDRANT_COLLECTION_NAME,
    embedding_dim=768,
    similarity="cosine",
    recreate_index=False,
    use_sparse_embeddings=True,
)
text_embedder = FastembedTextEmbedder(
    model=FASTEMBED_DENSE_MODEL,
)
sparse_text_embedder = FastembedSparseTextEmbedder(
    model=FASTEMBED_SPARSE_MODEL,
)
retriever = QdrantHybridRetriever(document_store=document_store)

pipeline = Pipeline()
pipeline.add_component("text_embedder", text_embedder)
pipeline.add_component("sparse_text_embedder", sparse_text_embedder)
pipeline.add_component("retriever", retriever)
pipeline.connect("text_embedder.embedding", "retriever.query_embedding")
```

```
pipeline.connect("sparse_text_embedder.sparse_embedding",
                 "retriever.query_sparse_embedding")
```

Listing B.9: Logica semplificata di taglio dinamico dei risultati.

```
scored_docs.sort(key=lambda x: x[0], reverse=True)
above_threshold = [
    d for score_value, d in scored_docs
    if score_value >= score_threshold
]

filtered = [above_threshold[0]]
for i in range(1, len(above_threshold)):
    prev = float(getattr(above_threshold[i - 1], "score", 0.0) or 0.0)
    cur = float(getattr(above_threshold[i], "score", 0.0) or 0.0)
    relative_drop = (prev - cur) / prev if prev > 0 else 1.0
    if relative_drop > gap_threshold:
        cutoff_reason = "relative-gap"
        break
    filtered.append(above_threshold[i])
```

Listing B.10: Formato del messaggio utente aumentato con contesto RAG.

```
final_message = (
    f"{message}\n\n"
    "____-CONTESTO-RAG-____\n"
    "Usa queste informazioni per rispondere.\n"
    f"{final_context}\n"
    "____-FINE-CONTESTO-____"
)
```

Listing B.11: Esempio di eventi NDJSON prodotti dalla chat standard.

```
{"type": "rag", "data": {"documents": [...], "cutoff_reason": "
    relative_gap"}}
{"type": "stream_start", "model": "qwen2.5:1.5b"}
{"type": "token", "content": "La"}
{"type": "token", "content": " gilda"}
{"type": "stream_end"}
```

B.4 Listati richiamati nel Capitolo 3: frontend e generazione bozza

Listing B.12: Esempio di eventi NDJSON prodotti dalla generazione bozza.

```
{ "type": "status", "message": "generation_started", "promptLength":  
  4200 }  
{ "type": "status", "message": "generation_in_progress" }  
{ "type": "draft", "source": "llm", "draft": { "name": "..." } }
```

Listing B.13: Chiamata TypeScript per generazione bozza senza retrieval.

```
const ragResponse = await generateRAGChatReply(prompt, [], {  
  model,  
  useRag: false,  
  jsonMode: true,  
  options: { num_predict: -1, timeout_seconds: 0 },  
});
```

Listing B.14: Chiamata TypeScript al retrieval RAG.

```
const response = await fetch(`${RAG_API_URL}/rag/retrieve`, {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({  
    query,  
    filters,  
    top_k: 20,  
    score_threshold: options?.scoreThreshold ?? 0.7,  
    max_documents: options?.maxDocuments ?? 10,  
  }),  
});
```

Bibliografia

Riferimenti Bibliografici

- [1] Meftun Akarsu, Recep Kaan Karaman e Christopher Mierbach. “From BM25 to Corrective RAG: Benchmarking Retrieval Strategies for Text-and-Table Documents”. In: *arXiv preprint arXiv:2604.01733* (2026). URL: <https://arxiv.org/abs/2604.01733>.
- [2] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint arXiv:2310.11511* (2023). URL: <https://arxiv.org/abs/2310.11511>.
- [3] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI Feedback”. In: *arXiv preprint arXiv:2212.08073* (2022). URL: <https://arxiv.org/abs/2212.08073>.
- [4] Devi Prasad Bal e Subhashree Puhan. “Benchmarking Retrieval Strategies for Biomedical Retrieval-Augmented Generation: A Controlled Empirical Study”. In: *arXiv preprint arXiv:2605.02520* (2026). URL: <https://arxiv.org/abs/2605.02520>.
- [5] Rishi Bommasani et al. “On the Opportunities and Risks of Foundation Models”. In: *arXiv preprint arXiv:2108.07258* (2021). URL: <https://arxiv.org/abs/2108.07258>.
- [6] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>.
- [7] Chi-Min Chan, Xinyu Chen, Rui Yin et al. “RQ-RAG: Learning to Rewrite Queries for Retrieval Augmented Generation”. In: *arXiv preprint* (2024).
- [8] Jiawei Chen et al. “Benchmarking Large Language Models in Retrieval-Augmented Generation”. In: *arXiv preprint arXiv:2309.01431* (2023). URL: <https://arxiv.org/abs/2309.01431>.
- [9] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: <https://arxiv.org/abs/2305.14314>.
- [10] Darren Edge et al. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv preprint arXiv:2404.16130* (2024). URL: <https://arxiv.org/abs/2404.16130>.
- [11] European Parliament e Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council — General Data Protection Regulation*. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.

- [12] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)*. Rapp. tecn. L 119. Official Journal of the European Union, 2016, pp. 1–88. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32016R0679>.
- [13] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997* (2024). URL: <https://arxiv.org/abs/2312.10997>.
- [14] Sepp Hochreiter e Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [15] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: (2022). URL: <https://arxiv.org/abs/2106.09685>.
- [16] Intré S.r.l. *10 pratiche per costruire una Learning Organization: il modello Intré*. <https://www.intre.it/2024/01/15/10-pratiche-per-costruire-una-learning-organization-il-modello-intre/>. Ultimo accesso: maggio 2026. 2024.
- [17] Intré S.r.l. *Archivio Gilde*. <https://www.intre.it/gilde/>. Ultimo accesso: aprile 2026. 2024.
- [18] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.
- [19] Albert Q. Jiang et al. “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825* (2023). URL: <https://arxiv.org/abs/2310.06825>.
- [20] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [21] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [22] Shih-Yang Liu et al. “DoRA: Weight-Decomposed Low-Rank Adaptation”. In: *International Conference on Machine Learning (ICML)*. 2024. URL: <https://arxiv.org/abs/2402.09353>.
- [23] Yury A. Malkov e Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [24] Qwen Team e Alibaba Cloud. *Qwen2 Technical Report*. 2024. URL: <https://arxiv.org/abs/2407.10671>.
- [25] Peter M. Senge. *The Fifth Discipline: The Art and Practice of the Learning Organization*. Edizione italiana: *La quinta disciplina*, Sperling & Kupfer, 1990. Doubleday/Currency, 1990.
- [26] Kurt Shuster et al. “Retrieval Augmentation Reduces Hallucination in Conversation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 3784–3803. URL: <https://arxiv.org/abs/2104.07567>.

- [27] Aparajita Sinha e Kunal Chakma. “NITATREC at TREC 2025: A Report on Exploring Sparse, Dense, and Hybrid Retrieval for Retrieval-Augmented Generation”. In: *Proceedings of the 34th Text REtrieval Conference (TREC 2025)*. Gaithersburg, Maryland, 2025. URL: <https://pages.nist.gov/trec-browser/trec34/rag/proceedings/>.
- [28] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023). URL: <https://arxiv.org/abs/2302.13971>.
- [29] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [30] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. URL: <https://arxiv.org/abs/2201.11903>.
- [31] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024). URL: <https://arxiv.org/abs/2401.15884>.
- [32] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [33] Shenglai Zeng, Jiankun Zhang, Pengfei He et al. “The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)”. In: *arXiv preprint arXiv:2402.16893* (2024). URL: <https://arxiv.org/abs/2402.16893>.

Sitografia

- [34] AIMultiple Research. *RAG Frameworks: LangChain vs LangGraph vs LlamaIndex*. Benchmark comparativo di 5 framework RAG su 100 query standardizzate. 2026. URL: <https://research.aimultiple.com/rag-frameworks/>.
- [35] Anthropic. *Introducing Claude 4*. 2025. URL: <https://www.anthropic.com/news/claude-4>.
- [36] Athenic AI. *LangChain vs LlamaIndex vs Haystack: RAG Framework Comparison*. 2025. URL: <https://getathenic.com/blog/langchain-vs-llamaindex-vs-haystack-rag-frameworks>.
- [37] deepset. *FastembedDocumentEmbedder*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/fastembeddocumentembedder>.
- [38] deepset. *FastembedSparseDocumentEmbedder*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/fastembedsparsedocumentembedder>.
- [39] deepset. *QdrantHybridRetriever*. Ultimo accesso: 25 maggio 2026. 2026. URL: <https://docs.haystack.deepset.ai/docs/qdranthybridretriever>.
- [40] deepset GmbH. *Haystack: Open Source NLP Framework to Build Production-Ready LLM Applications*. 2023. URL: <https://haystack.deepset.ai>.
- [41] Docker Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/>.
- [42] FastAPI Contributors. *FastAPI Documentation*. 2024. URL: <https://fastapi.tiangolo.com/>.

- [43] Google DeepMind. *Gemini 3 Pro*. 2025. URL: <https://deepmind.google/en/models/gemini/pro/>.
- [44] Nirant Kasliwal. *pgvector vs Qdrant: Results from the 1M OpenAI Benchmark*. 2024. URL: <https://nirantk.com/writing/pgvector-vs-qdrant/>.
- [45] LangChain AI. *LangGraph: Build Resilient Language Agents as Graphs*. 2024. URL: <https://langchain-ai.github.io/langgraph/>.
- [46] Meta AI. *Llama 4: The Next Generation of Open Foundation Models*. 2025. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [47] Mistral AI. *Introducing Mistral 3*. 2025. URL: <https://mistral.ai/news/mistral-3>.
- [48] Ollama Contributors. *Ollama: Get Up and Running With Large Language Models Locally*. 2023. URL: <https://ollama.com>.
- [49] OpenAI. *Introducing GPT-5*. 2025. URL: <https://openai.com/index/introducing-gpt-5/>.
- [50] OpenAI. *Learning to Reason with LLMs*. 2024. URL: <https://openai.com/index/learning-to-reason-with-llms/>.
- [51] Pelican. *Haystack vs LangChain: Explained*. 2025. URL: <https://pelican.io/blog/haystack-vs-langchain/>.
- [52] Ethan Perez. *RAG Evaluation Series: Validating the RAG Performance of LangChain vs Haystack*. Tonic.ai Blog. 2024. URL: <https://www.tonic.ai/blog/rag-evaluation-series-validating-the-rag-performance-of-langchain-vs-haystack>.
- [53] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/>.
- [54] Pydantic Contributors. *Pydantic Documentation*. 2024. URL: <https://docs.pydantic.dev/>.
- [55] Qdrant Contributors. *Qdrant: Vector Database for the Next Generation of AI Applications*. 2023. URL: <https://qdrant.tech>.
- [56] Qdrant Team. *Vector Search Benchmarks*. Benchmark ufficiale Qdrant su dataset ann-benchmarks; aggiornato gennaio/giugno 2024. 2024. URL: <https://qdrant.tech/benchmarks/>.
- [57] Qwen Team e Alibaba Cloud. *Qwen3 Technical Report*. 2025. URL: <https://qwenlm.github.io/blog/qwen3/>.
- [58] Leonard Richardson. *Beautiful Soup Documentation*. 2024. URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
- [59] Tiger Data. *Pgvector vs. Qdrant: Benchmarking Open-Source Vector Databases for AI at Scale*. Benchmark su 50M embedding Cohere a 768 dimensioni con fork ANN-Benchmarks. 2025. URL: <https://www.tigerdata.com/blog/pgvector-vs-qdrant>.
- [60] Vercel. *Next.js Documentation*. 2024. URL: <https://nextjs.org/docs>.